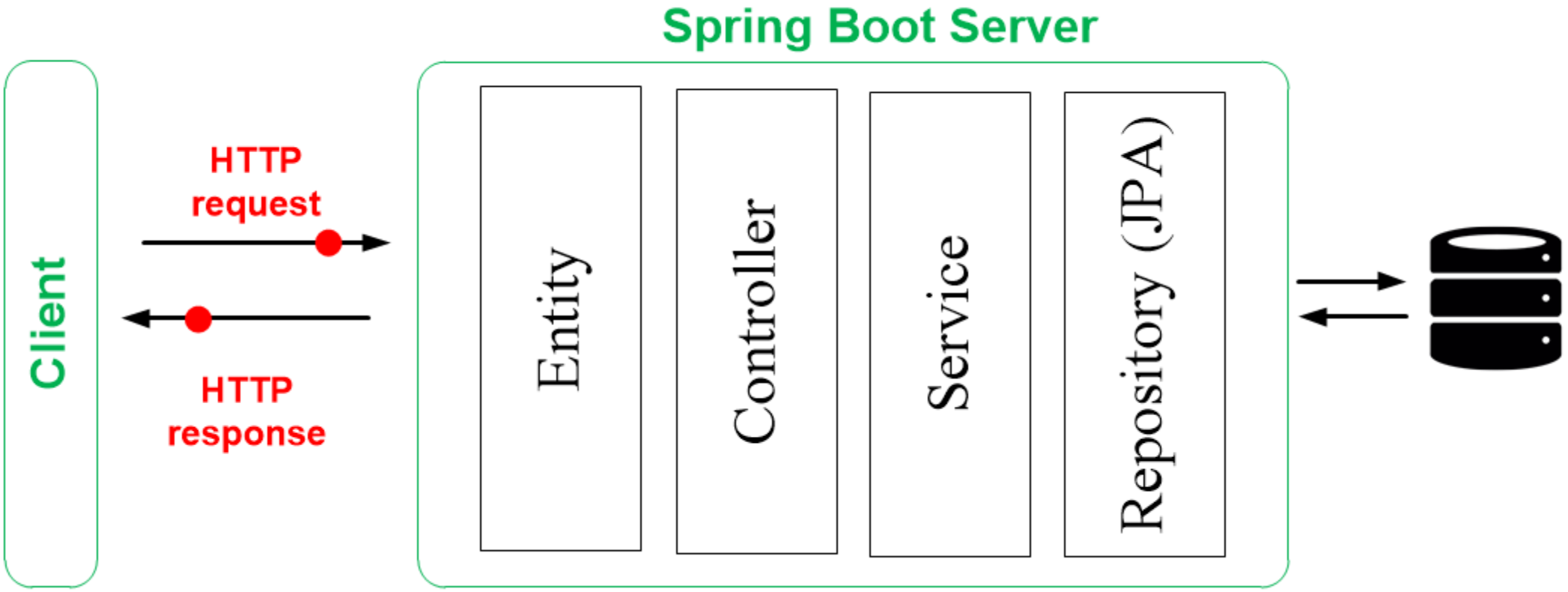
Enterprise System Integration (MTAT.03.229)

LECTURE 8: VUE.JS

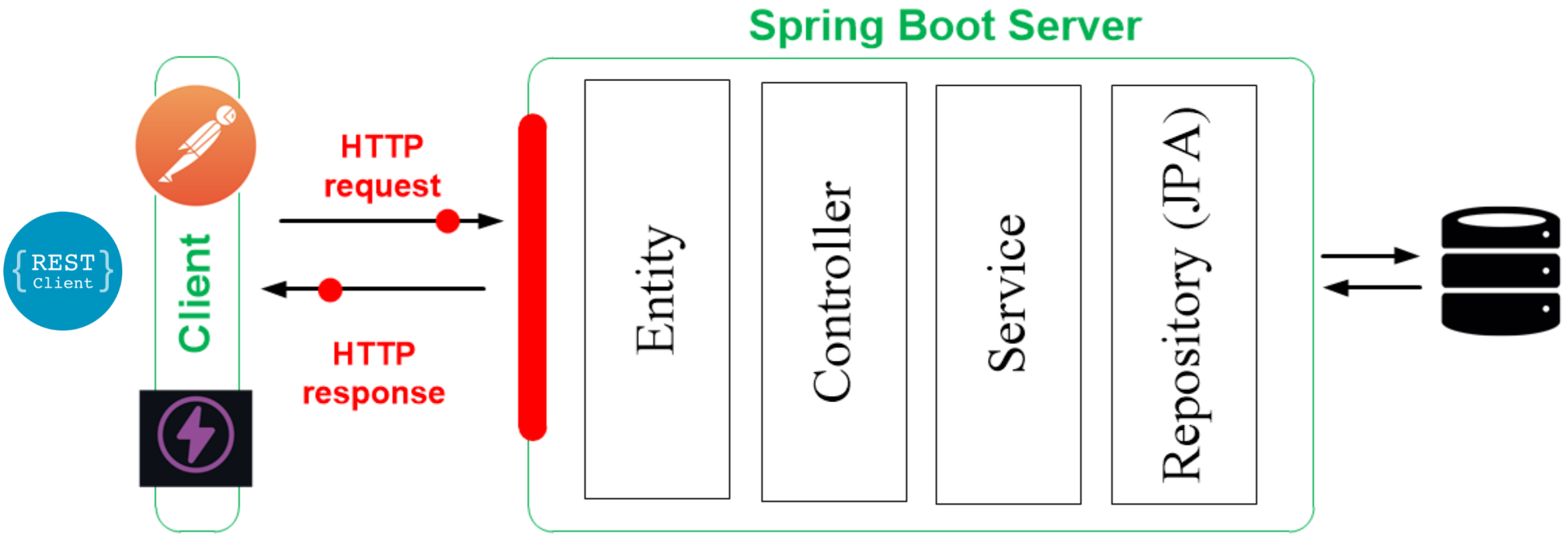
MOHAMAD GHARIB

UNIVERSITY OF TARTU

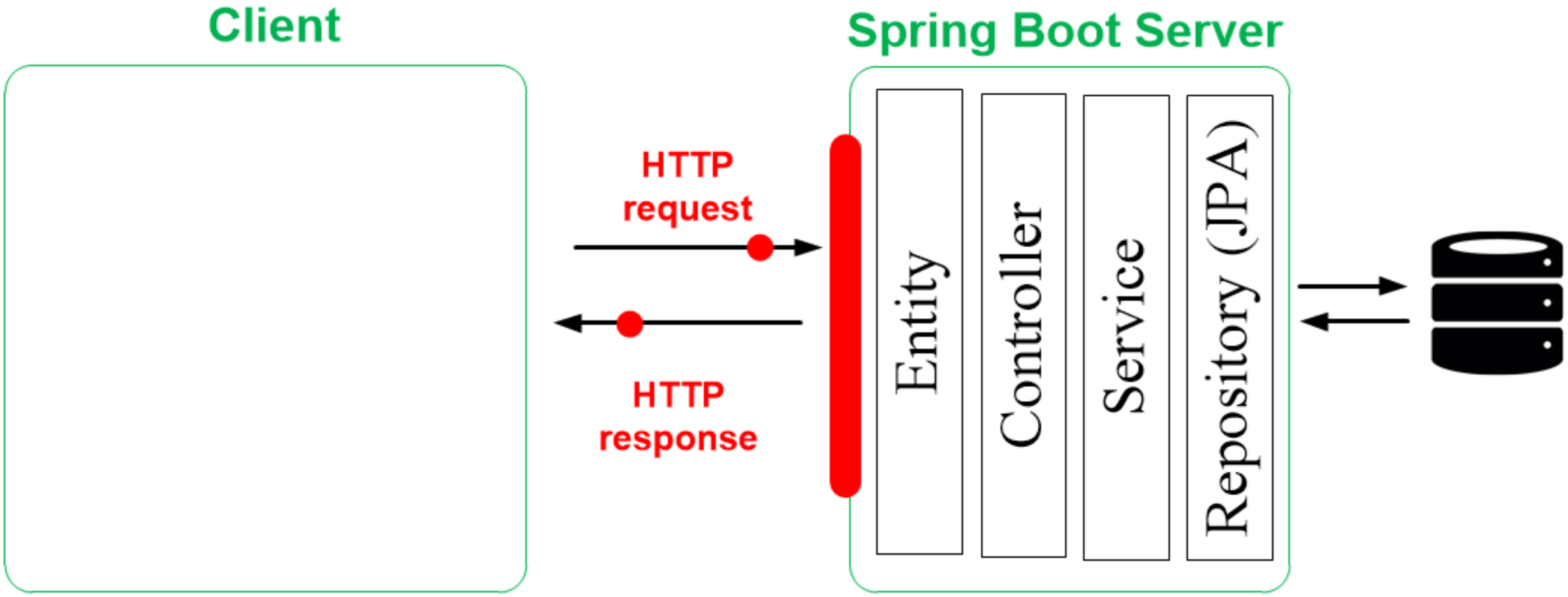
Spring Boot Server - recap



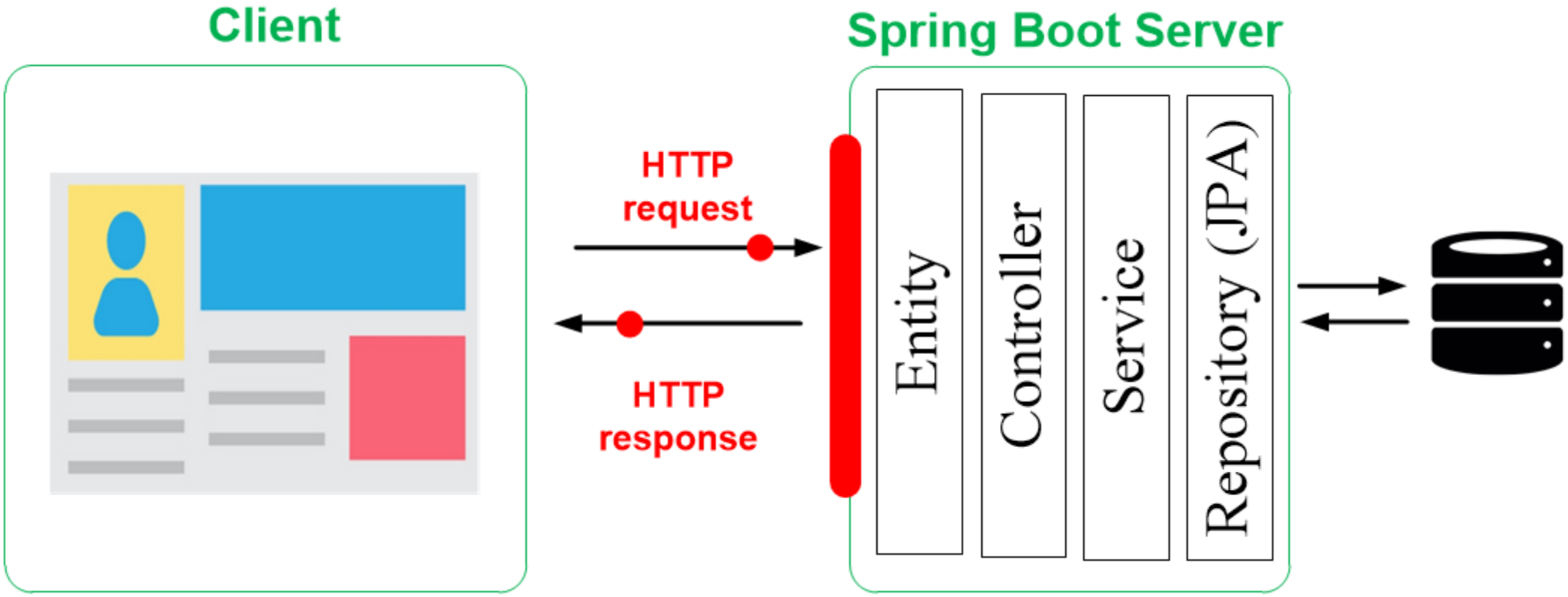
Spring Boot Server - recap



The Client side



The Client side



Vue.js

Vue.js

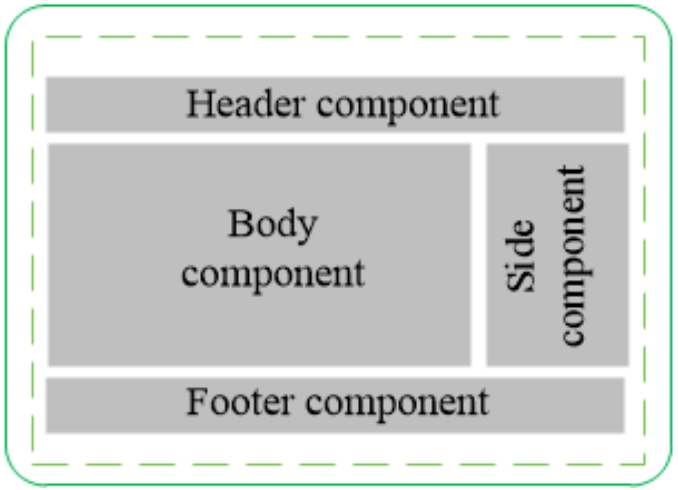
Vue is a progressive ***framework*** for building user interfaces.

Vue front-end JavaScript framework.

Vue is capable of powering sophisticated ***Single-Page Applications (SPA)***, when used in combination with modern tooling and supporting ***libraries***.

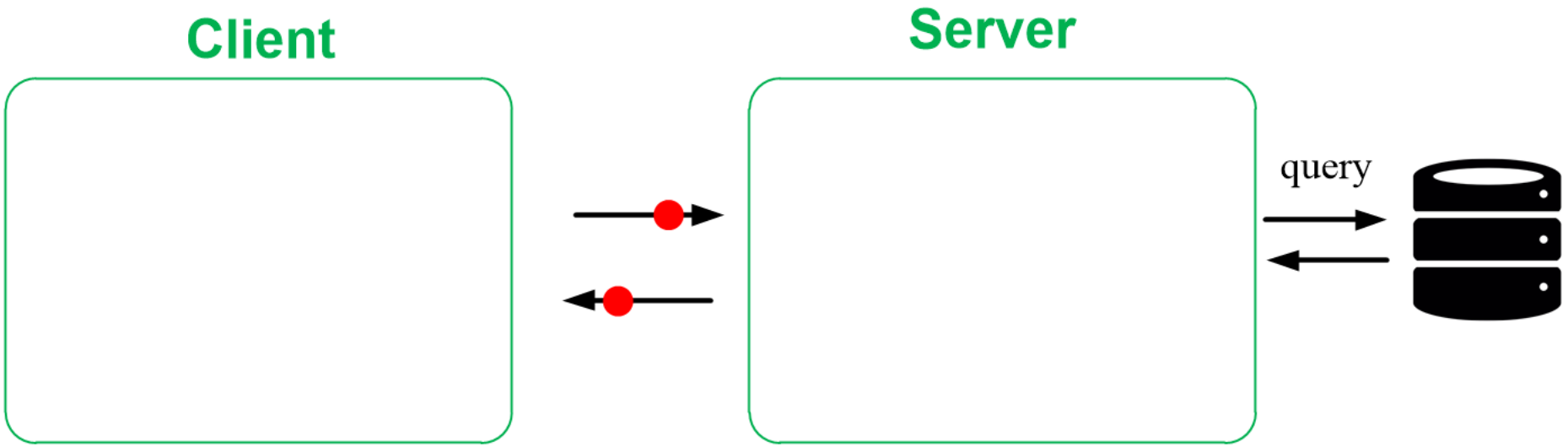
Vue can be used also to create ***Standalone wedges***

Single-Page Applications (SPA)

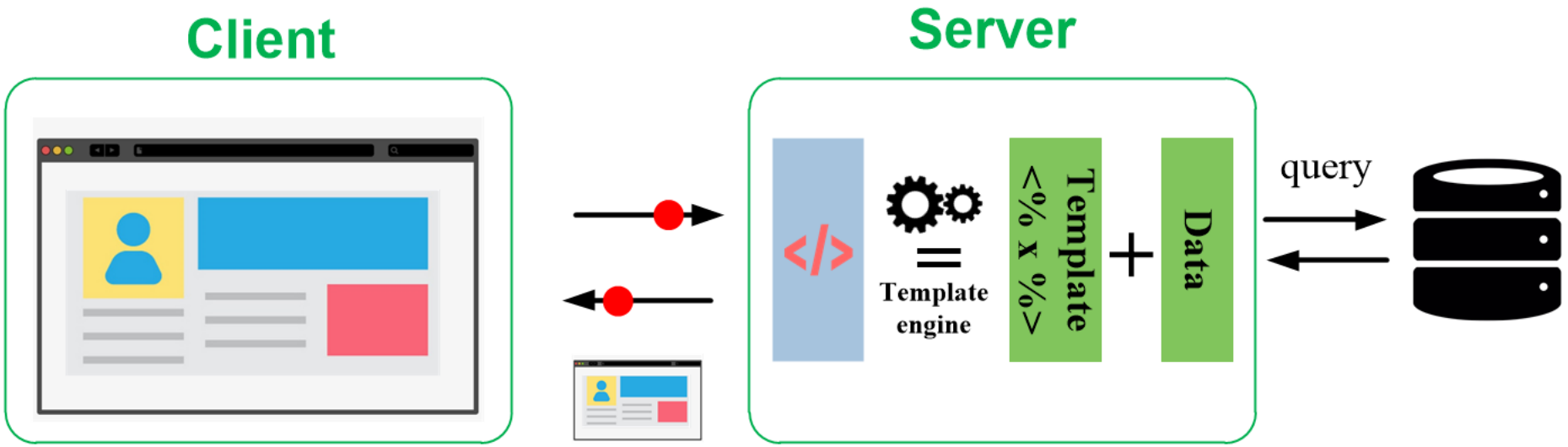


- Light weight
- Routing
- Reusable components
- Directives
- Event Handling
- Watched and computed properties
- Unit and Integration testing
- Command Line Interface (CLI)

Single-Page Applications (SPA)? Client-side vs. Server-side rendering



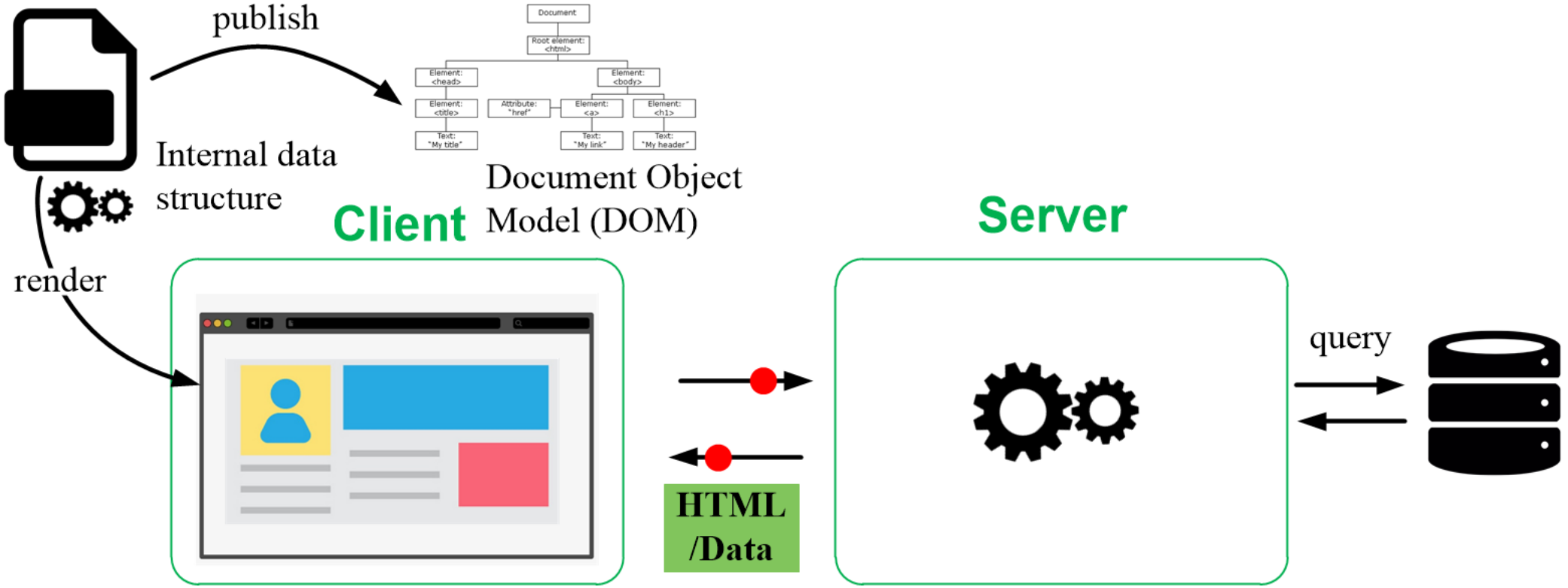
Single-Page Applications (SPA)? Client-side vs. Server-side rendering



In ***server-side*** rendering, the server ***compiles*** everything and ***delivers*** a fully populated HTML page to the ***client***.

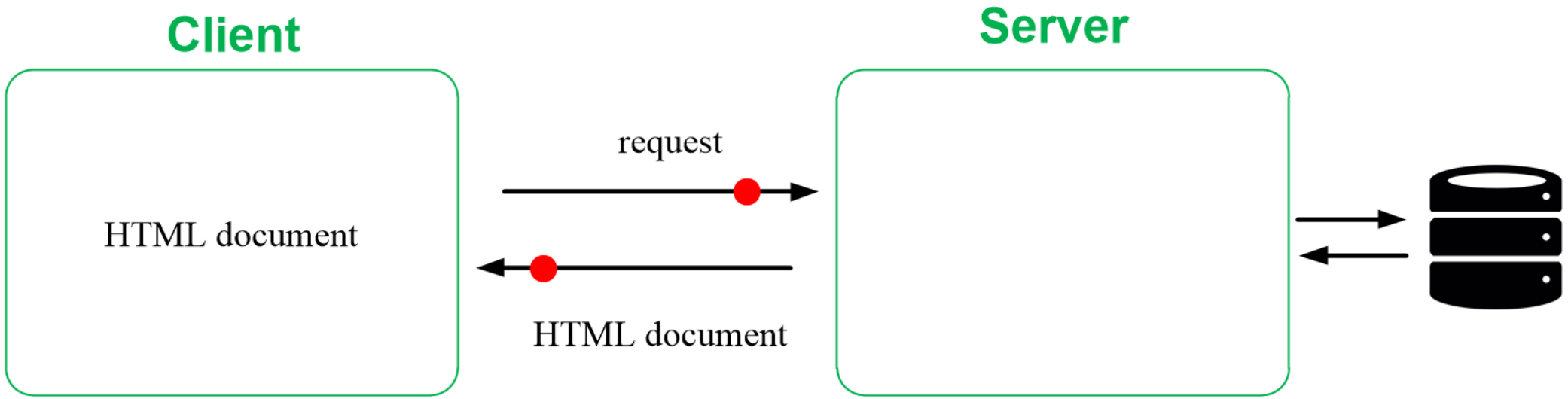
This is done ***every time*** you navigated to a new route/page.

Single-Page Applications (SPA)? Client-side vs. Server-side rendering

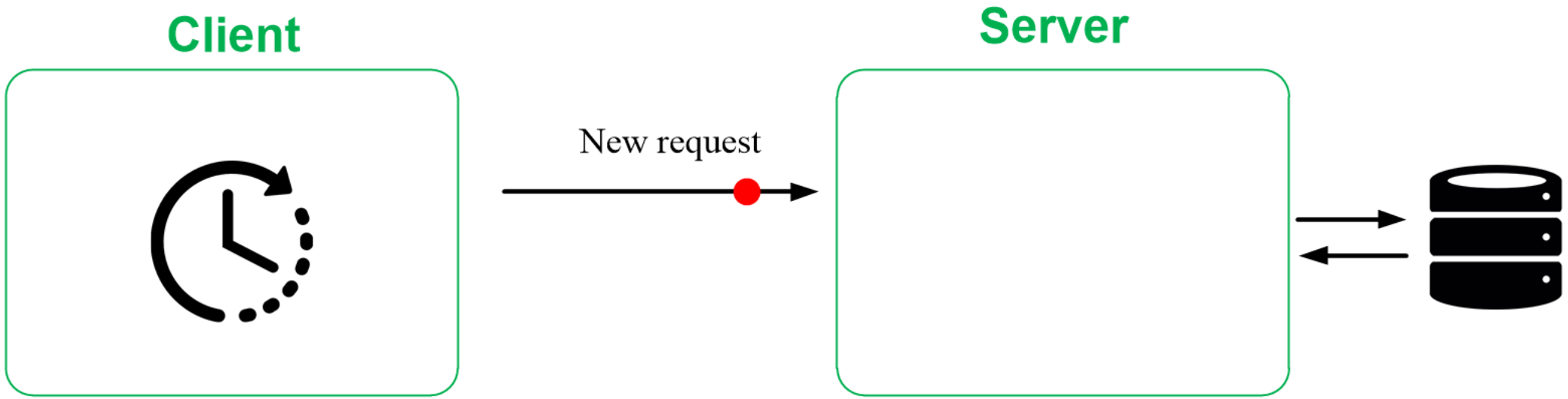


In ***client-side*** rendering, the ***server*** will deliver a single HTML file without "any content" to the ***client***. Then, the ***client*** fetches and compile everything before ***rendering*** the content.

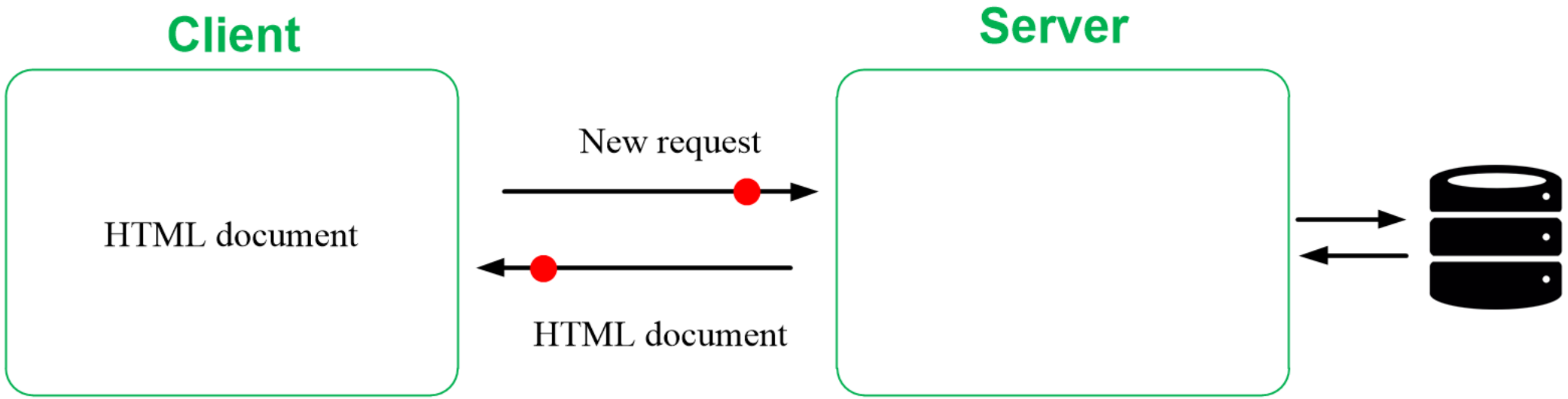
Non Single-Page Applications (SPA)



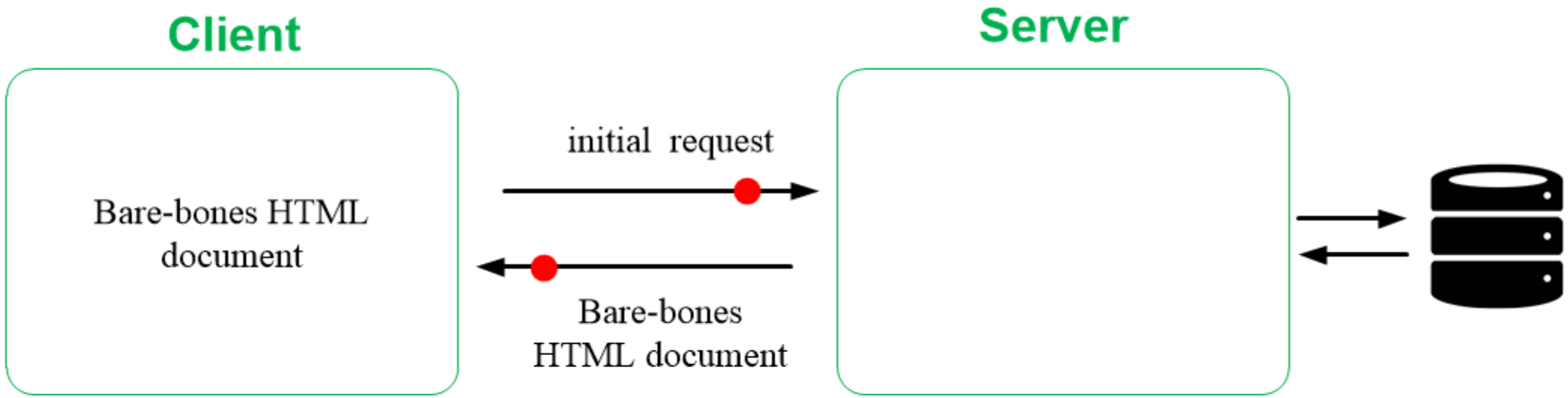
Non Single-Page Applications (SPA)



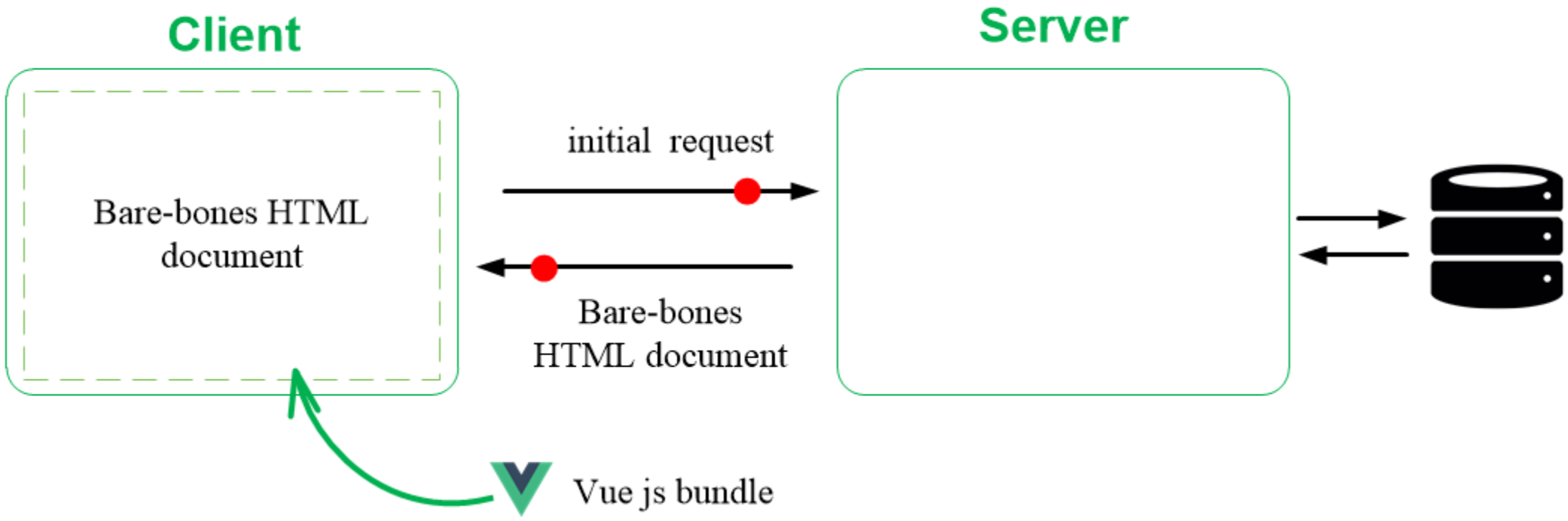
Non Single-Page Applications (SPA)



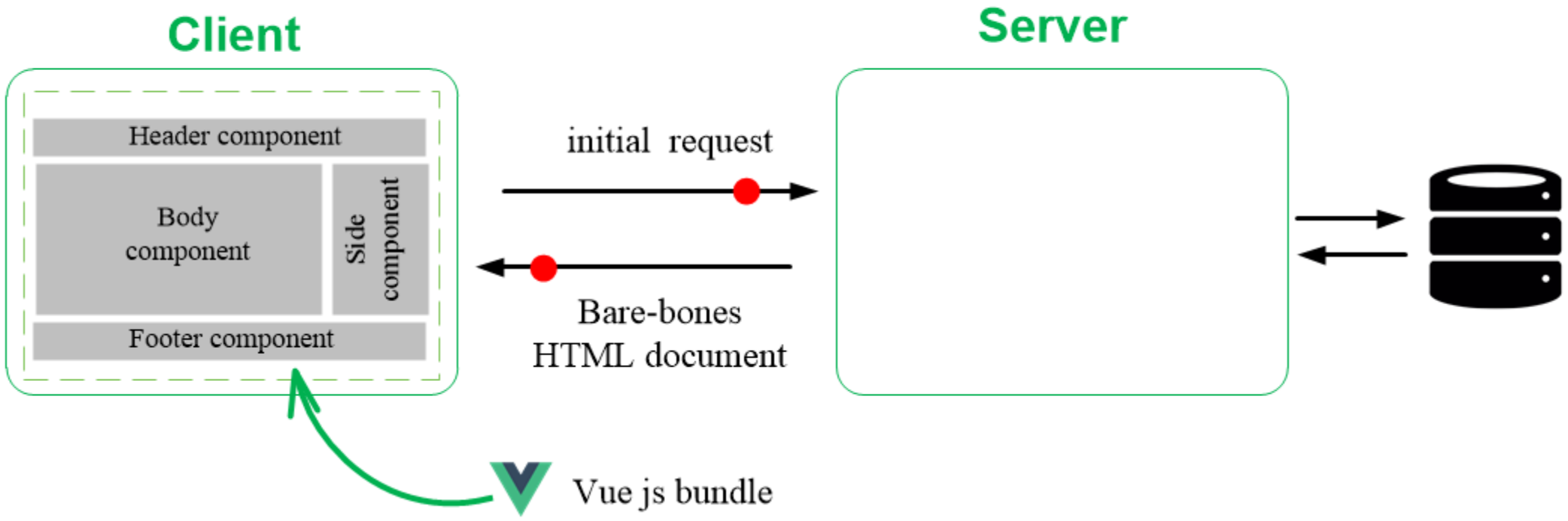
Vue.js - Single-Page Applications (SPA)



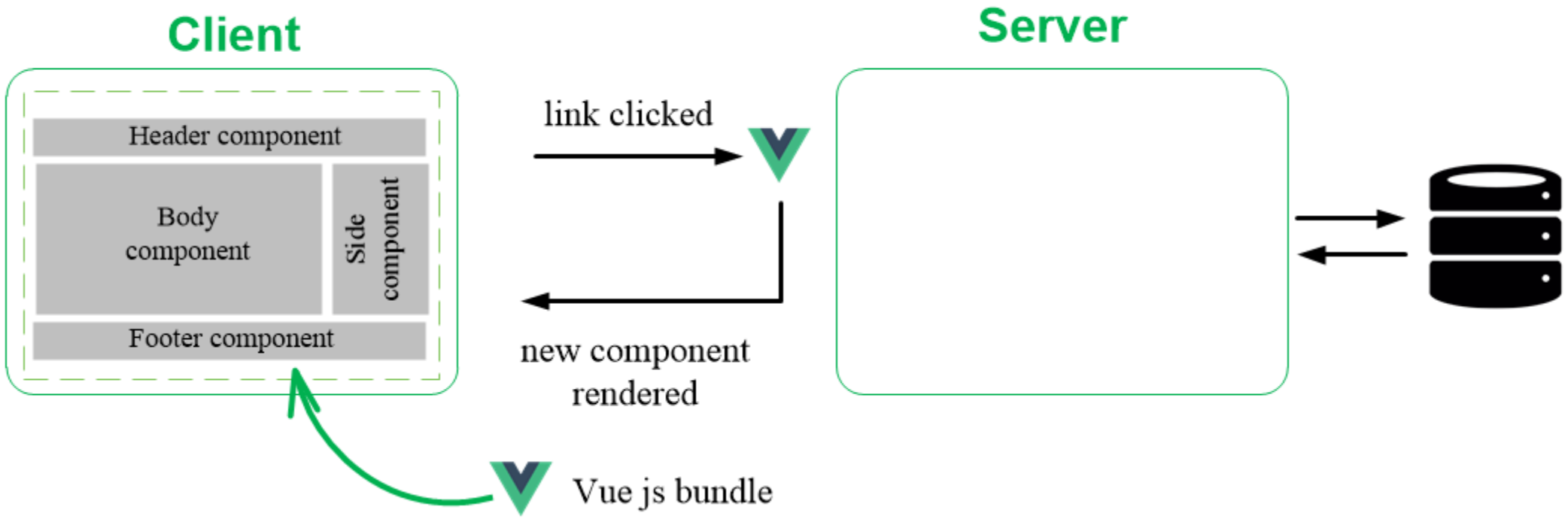
Vue.js - Single-Page Applications (SPA)



Vue.js - Single-Page Applications (SPA)

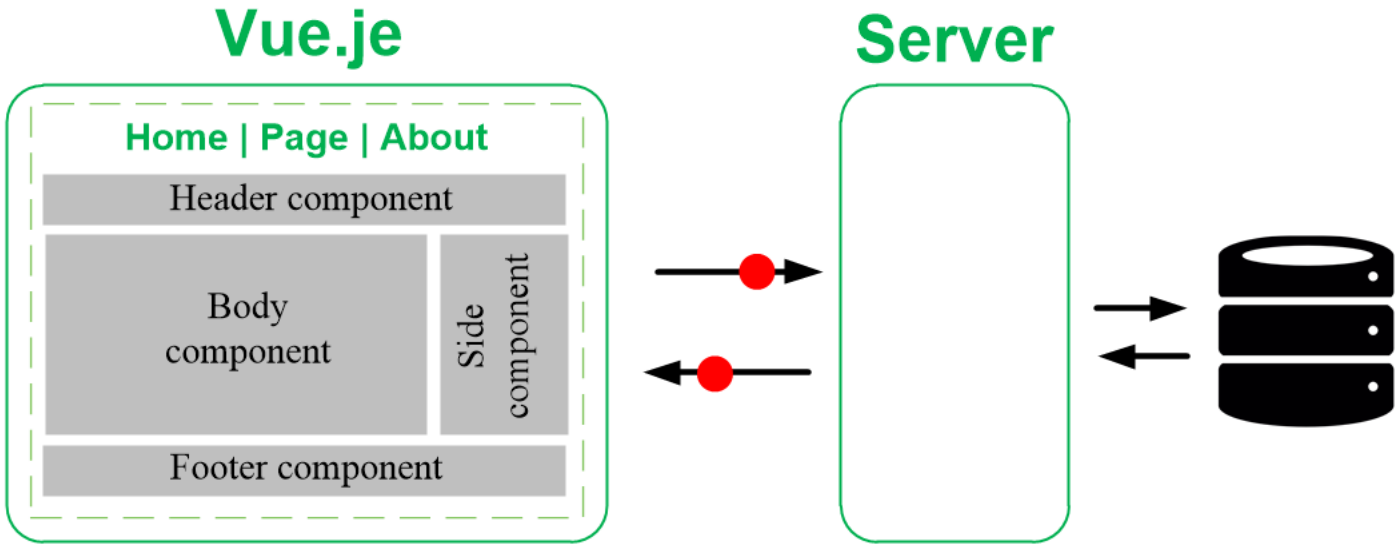


Vue.js - Single-Page Applications (SPA)

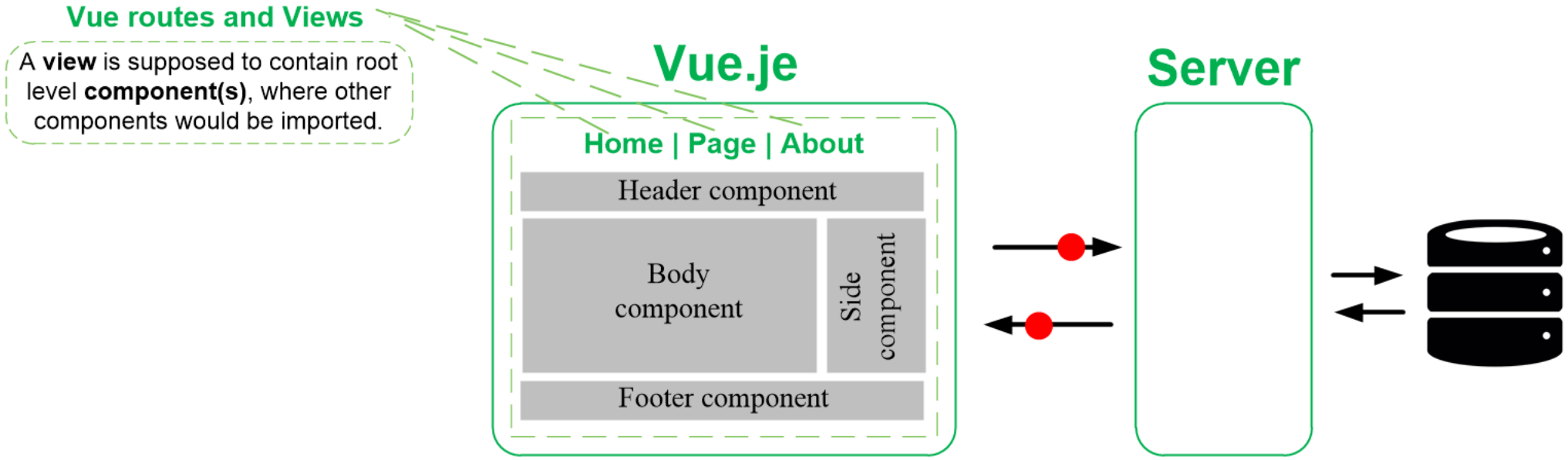


Vue.js in nutshell

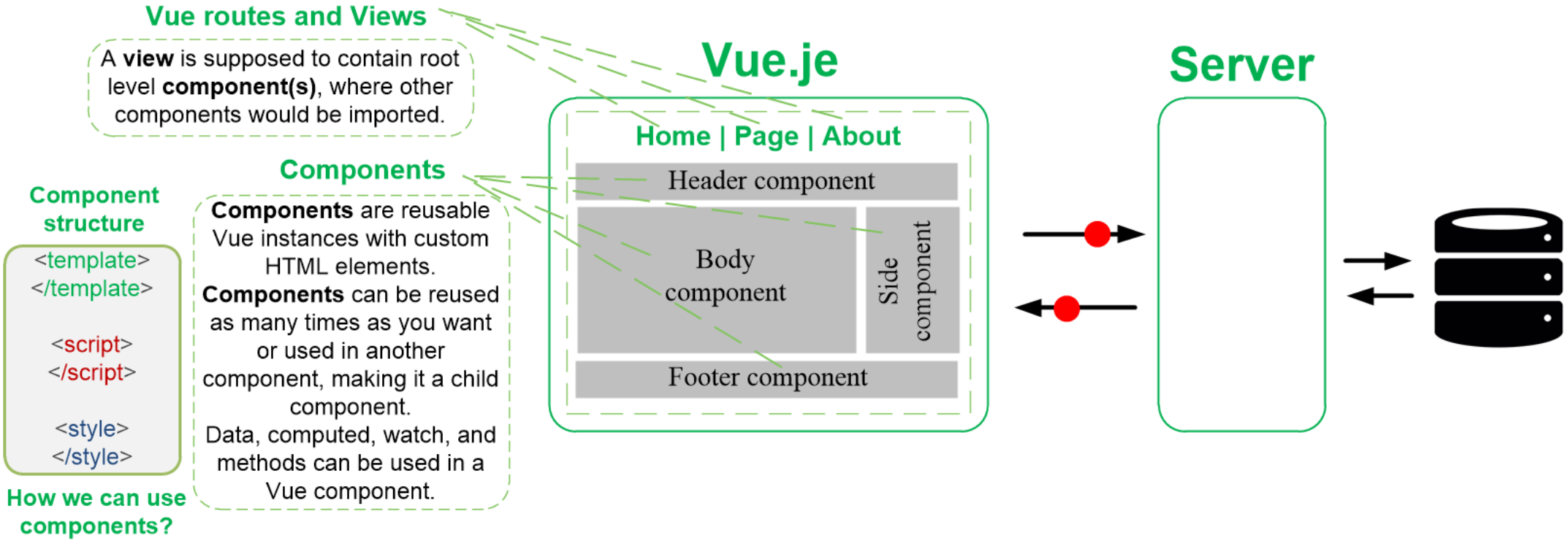
Vue.js in nutshell



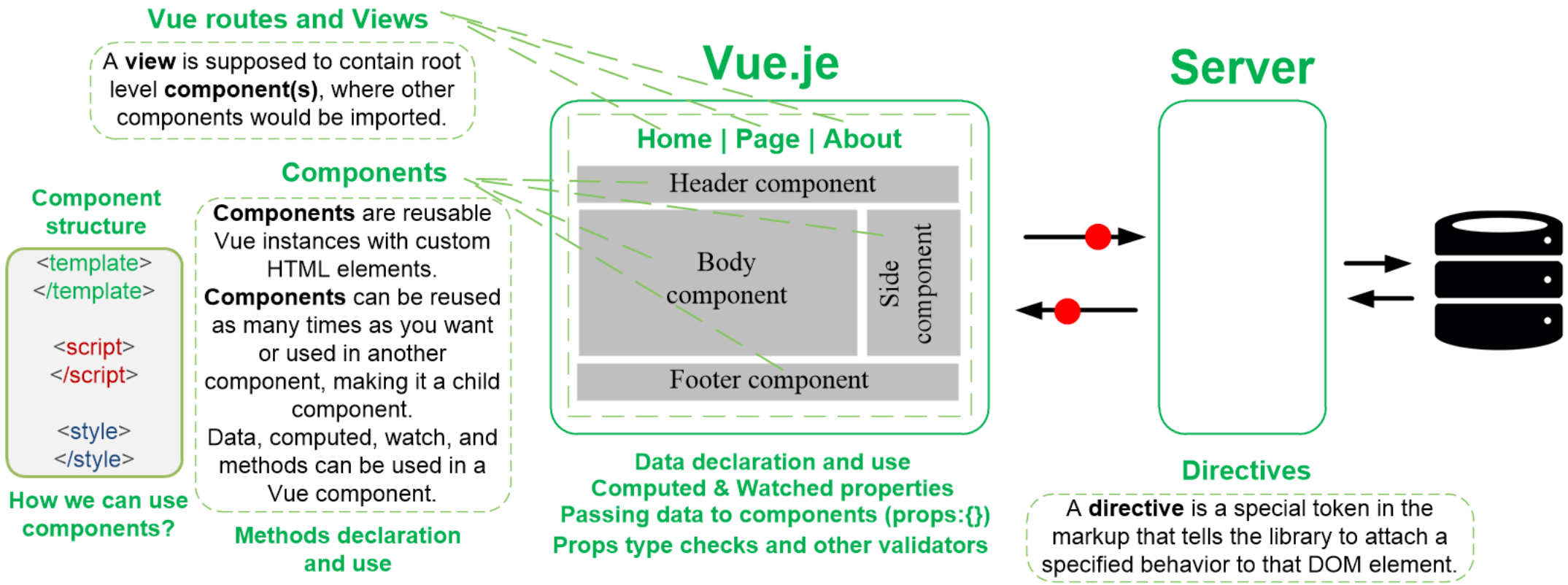
Vue.js in nutshell



Vue.js in nutshell



Vue.js in nutshell



Vue CLI project

Vue CLI project

Home | About

Welcome to Your Vue.js App

For a guide and recipes on how to configure / customize this project, check out the vue-cli documentation.

Installed CLI Plugins

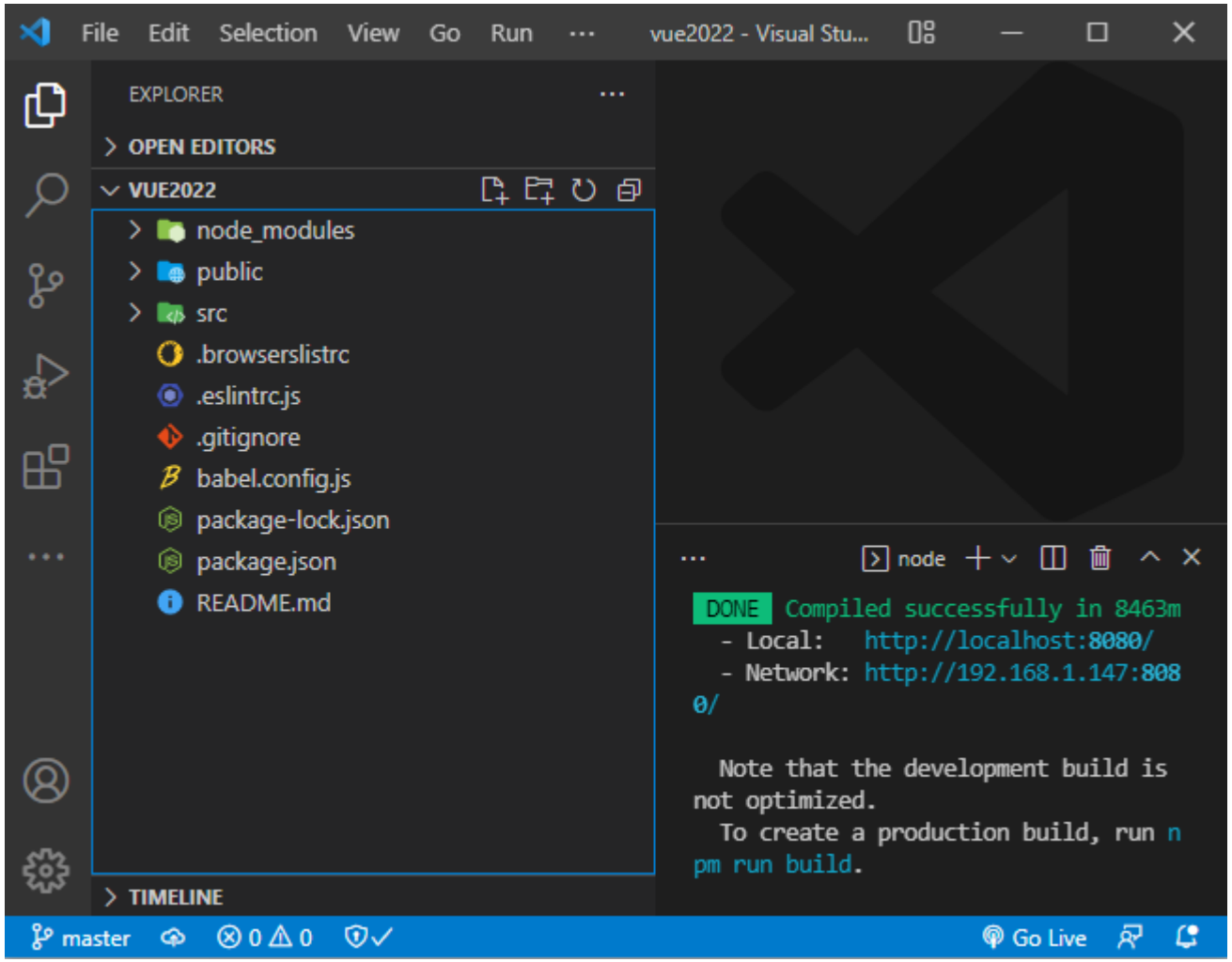
babel router vuex eslint

Essential Links

Core Docs Forum Community Chat Twitter News

Ecosystem

vue-router vuex vue-devtools vue-loader awesome-vue



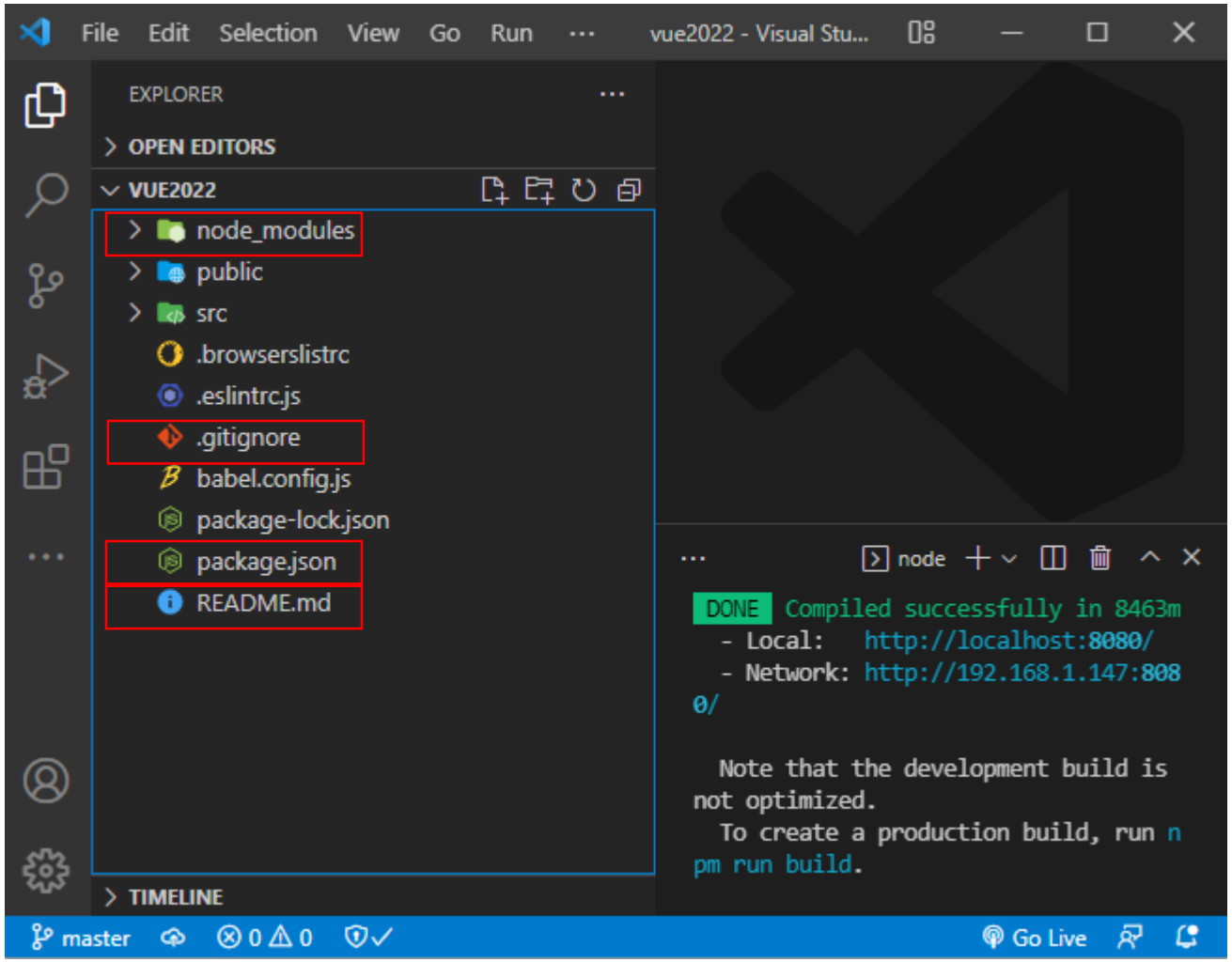
Vue CLI project

node_modules directory contains the full set of dependencies for the application.

package.json file used by the application to know which node_modules to get.

.gitignore contains files that will not be committed to the repo during git push or commit operations.

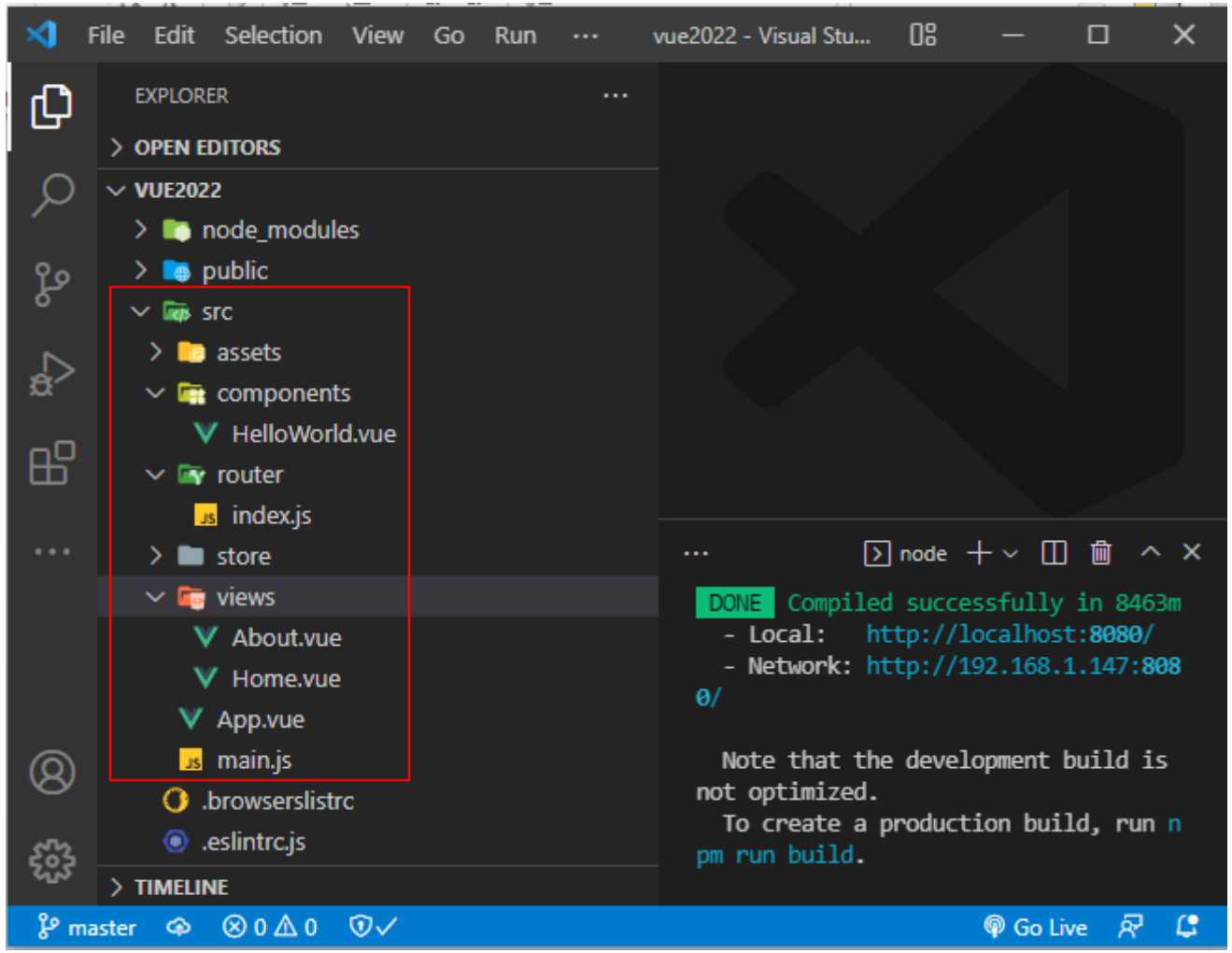
README.md basic readme file that contains essential commands to install and run your project.



Vue CLI project

src directory contains:

- **assets**
- **components**
  - HellowWorld.vue
- **router**
  - index.js
- **store**
- **views**
  - About.vue
  - Home.vue
- **App.vue**
- **main.js**

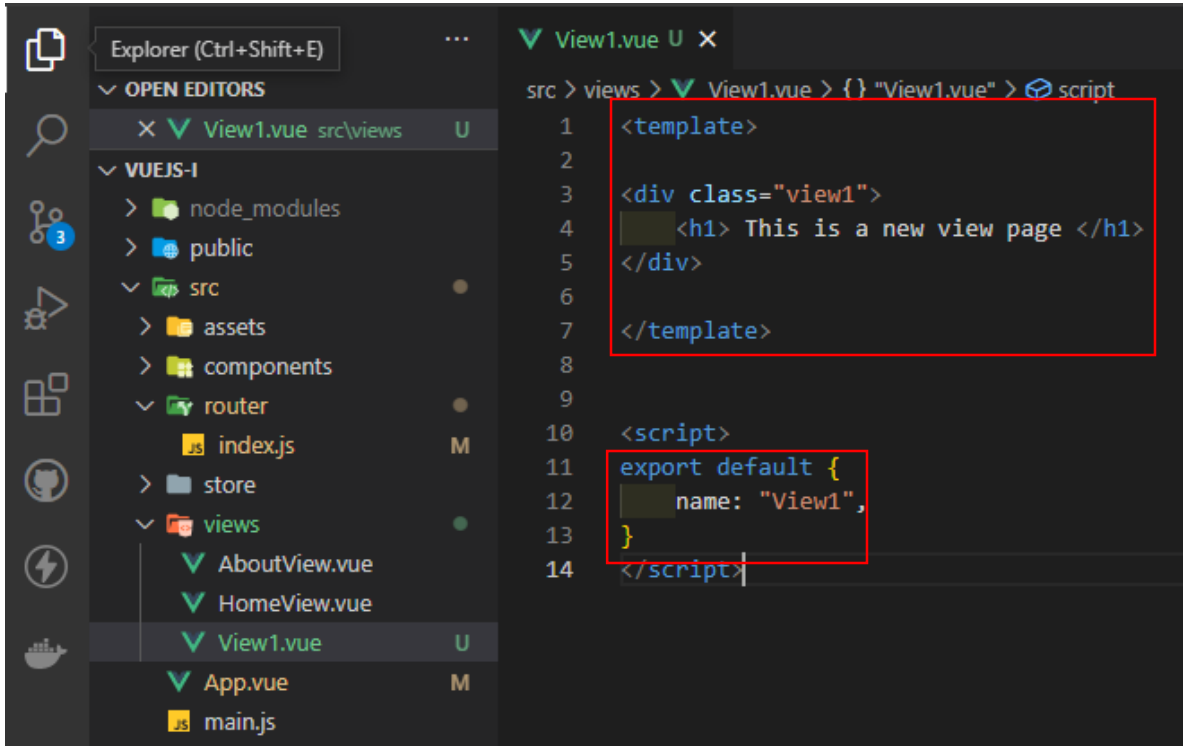


Vue Routers & Views

Views & Vue Router

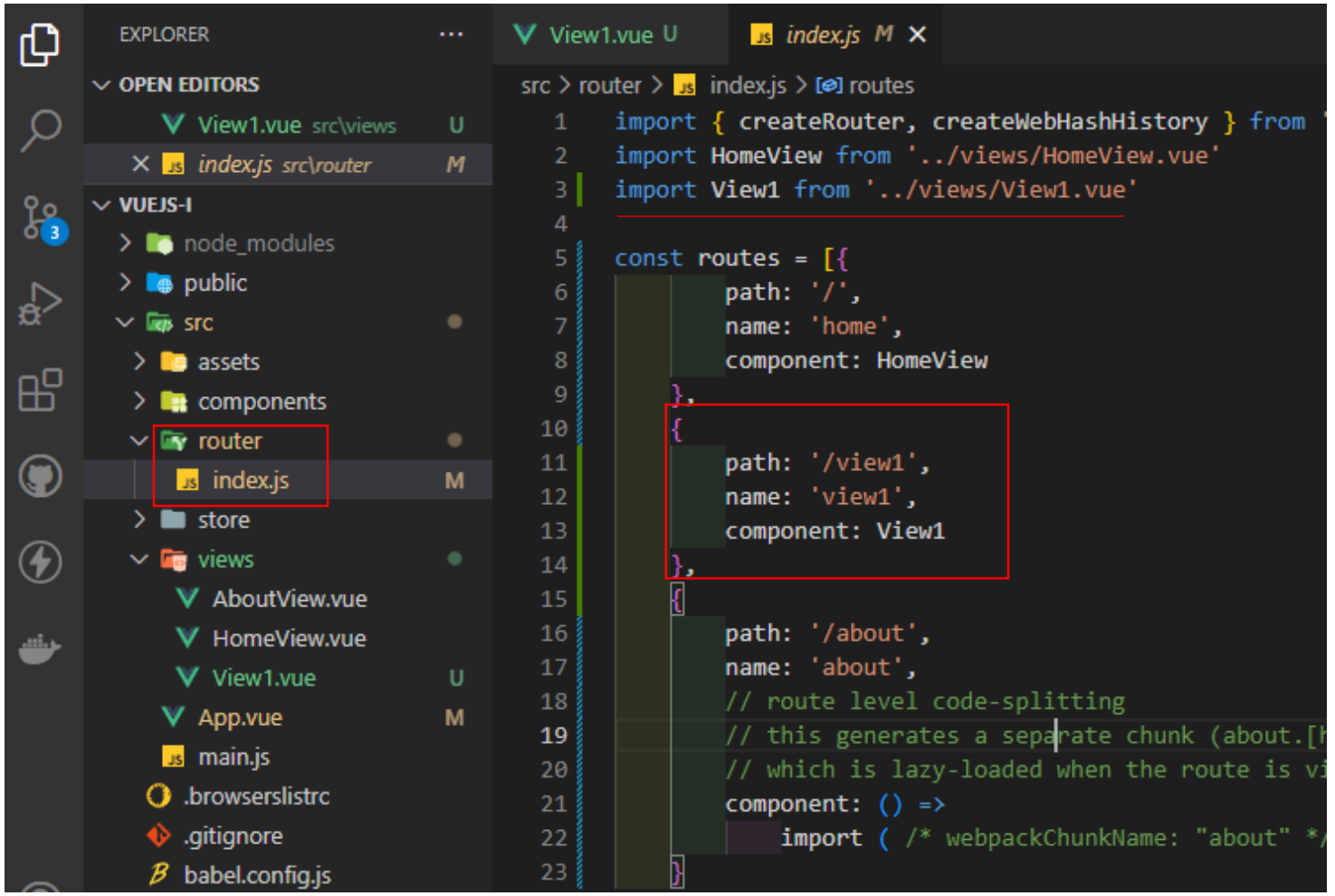
1. Create a View

- In the Views directory, create a new file (e.g., View1.vue).
- Export the view.



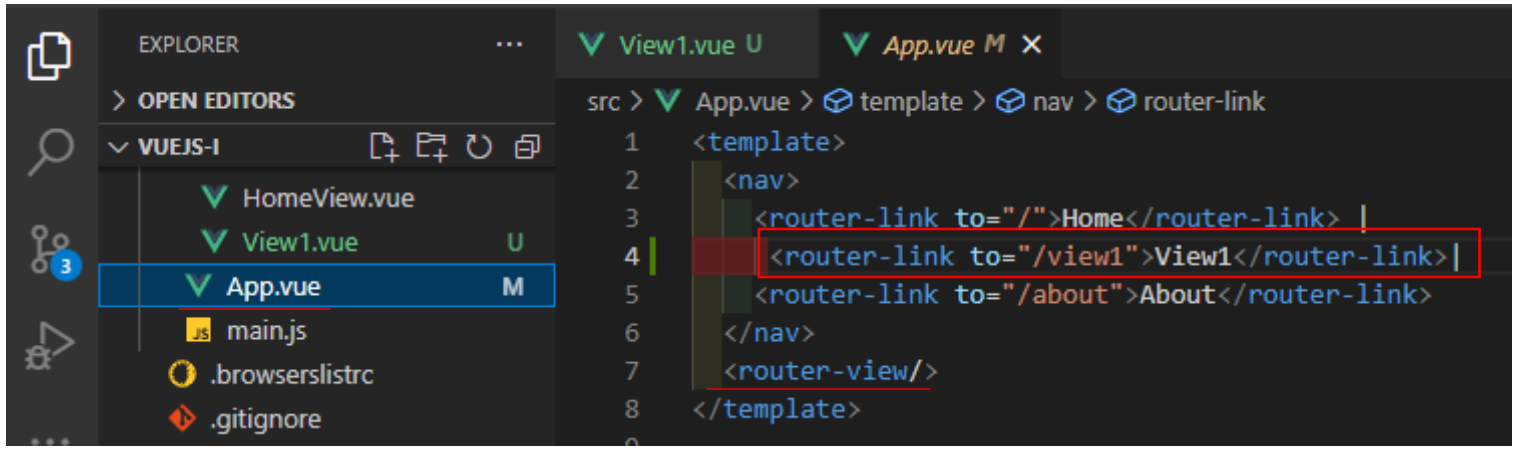
Views & Vue Router

1. **Create a View**
   - In the Views directory, create a new file (e.g., View1.vue).
   - Export the view.
2. **Create a Vue Route**
   - In the route/index.js, update the routes array by adding the new route.
   - Register/Import the new view.



Views & Vue Router

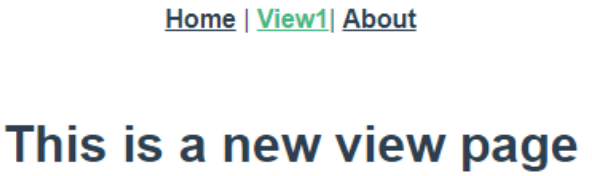
1. **Create a View**
   - In the Views directory, create a new file (e.g., View1.vue).
   - Export the view.
2. **Create a Vue Route**
   - In the route/index.js, update the routes array by adding the new route.
   - Register/Import the new view.
3. **Create the router-link**
   - In App.vue, add the **route link**.

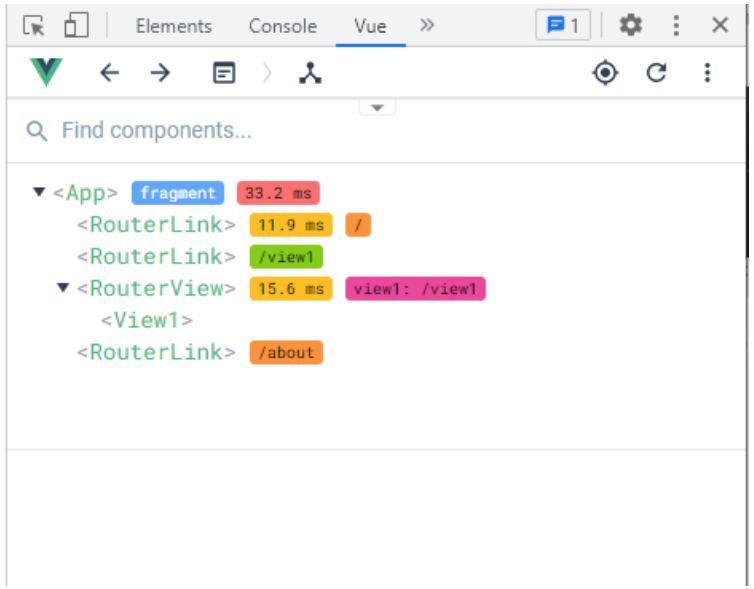


<router-view /> is a functional component that renders the matched component for the given path.

Views & Vue Router

1. **Create a View**
   - In the Views directory, create a new file (e.g., View1.vue).
   - Export the view.
2. **Create a Vue Route**
   - In the route/index.js, update the routes array by adding the new route.
   - Register/Import the new view.
3. **Create the router-link**
   - In App.vue, add the **route link**.



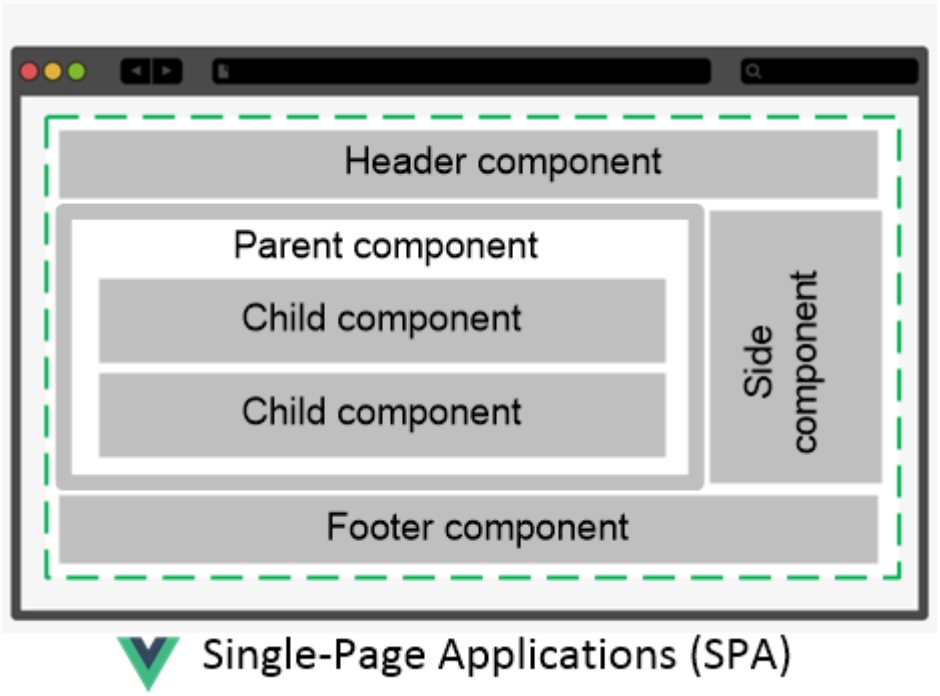


To test how SPA works: In the browser, Right-click -> inspect open the **network tab** and try to navigate between the Vue app routes. You should notice that there are no traffic.

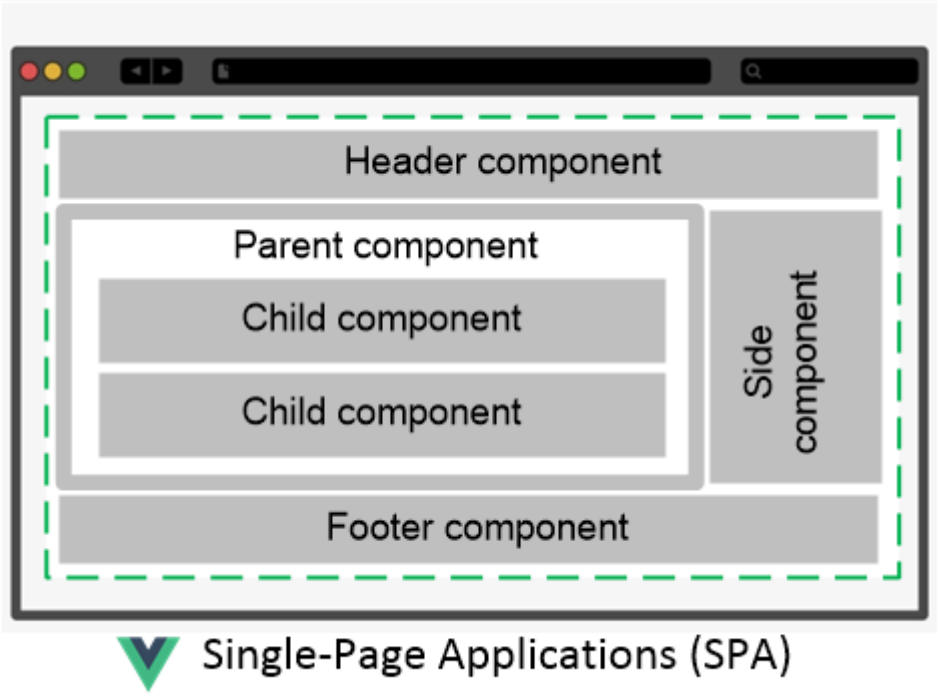
Components

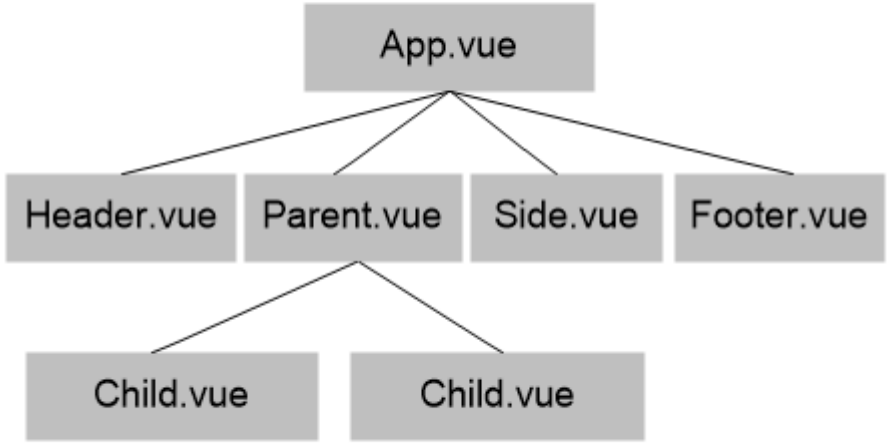
Single File Components (SFC)

Components


Header component
Parent component
Child component
Child component
Side component
Footer component
Single-Page Applications (SPA)

Components

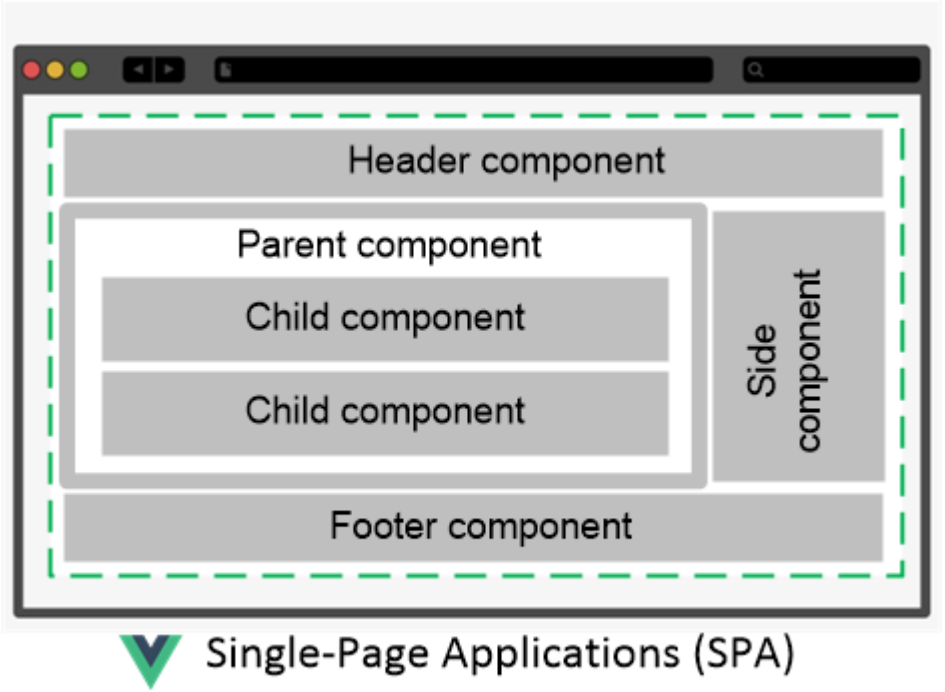

Header component
Parent component
Child component
Child component
Side component
Footer component
Single-Page Applications (SPA)


App.vue
Header.vue
Parent.vue
Side.vue
Footer.vue
Child.vue
Child.vue

Components

Components (SFC) are reusable Vue instances with a name.

We can use this component as a custom element inside a root Vue instance.



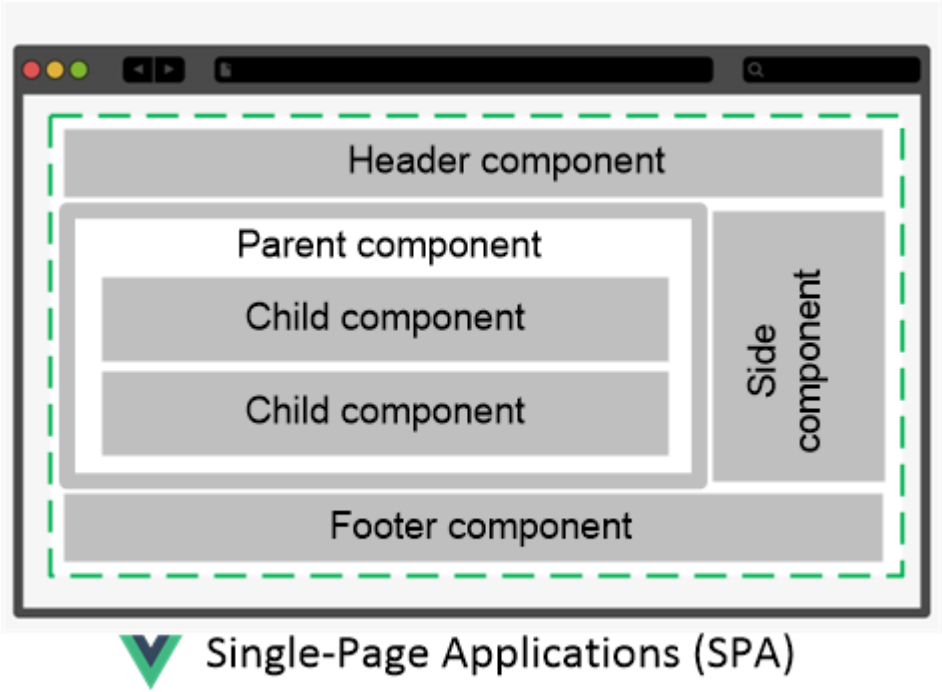
Components

Components (SFC) are reusable Vue instances with a name.

We can use this component as a custom element inside a root Vue instance.

A Vue component is a special file format that allows us to encapsulate the **template**, **logic**, and **styling** in a single file.

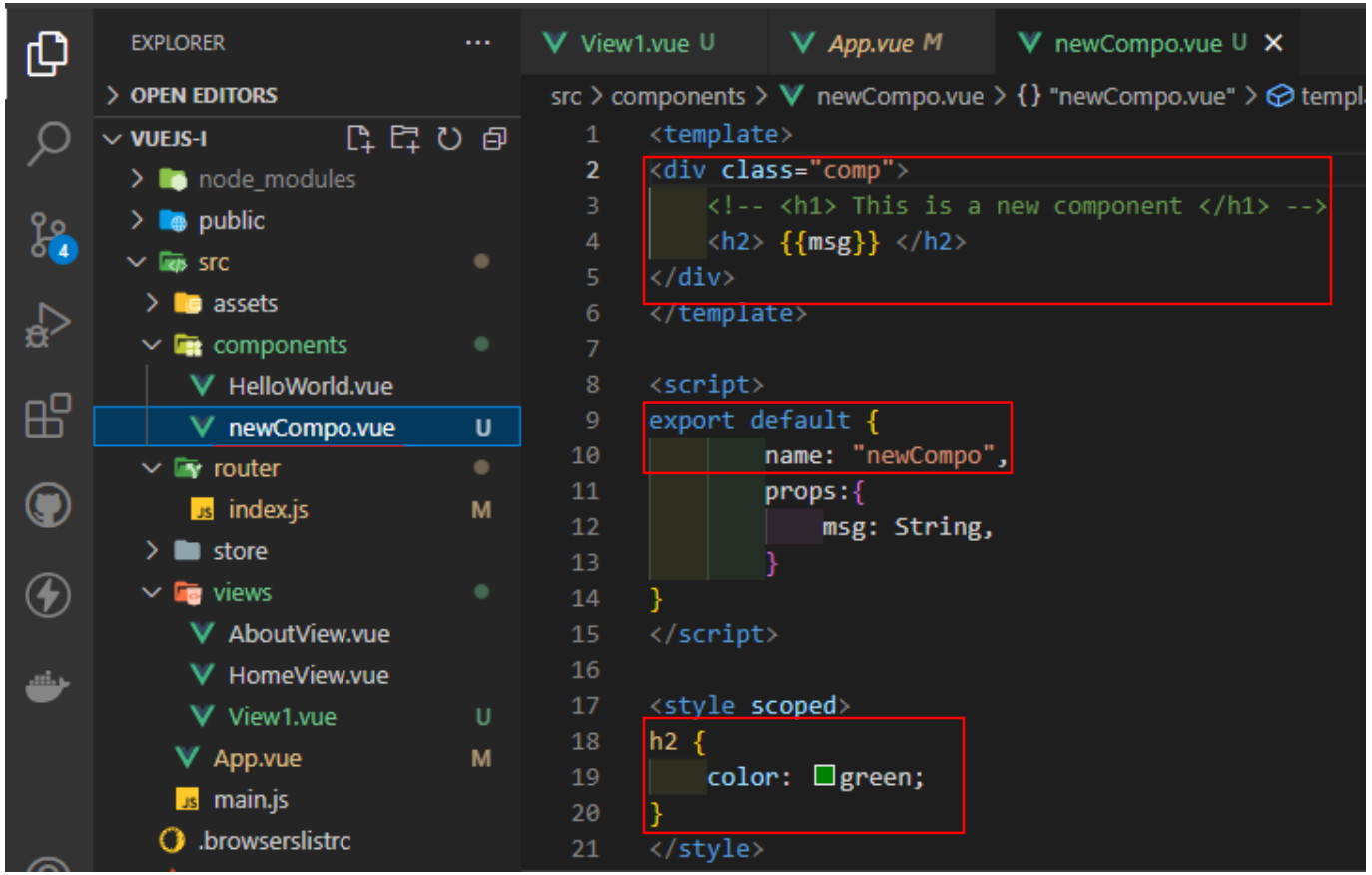
```
<template>
    <HTML>
</template>

<script>
    JavaScript
</script>

<style>
    CSS
</style>
```



Components

1. **Create a component**
   - In **components** directory, create a new file (e.g., newCompo.vue).
   - Add the structure **<template>**, **<script>,** and **<style>**.
2. Export the **component**, i.e., make it available to be used by other **components** and **views**.

export default {name: “newCompo", }



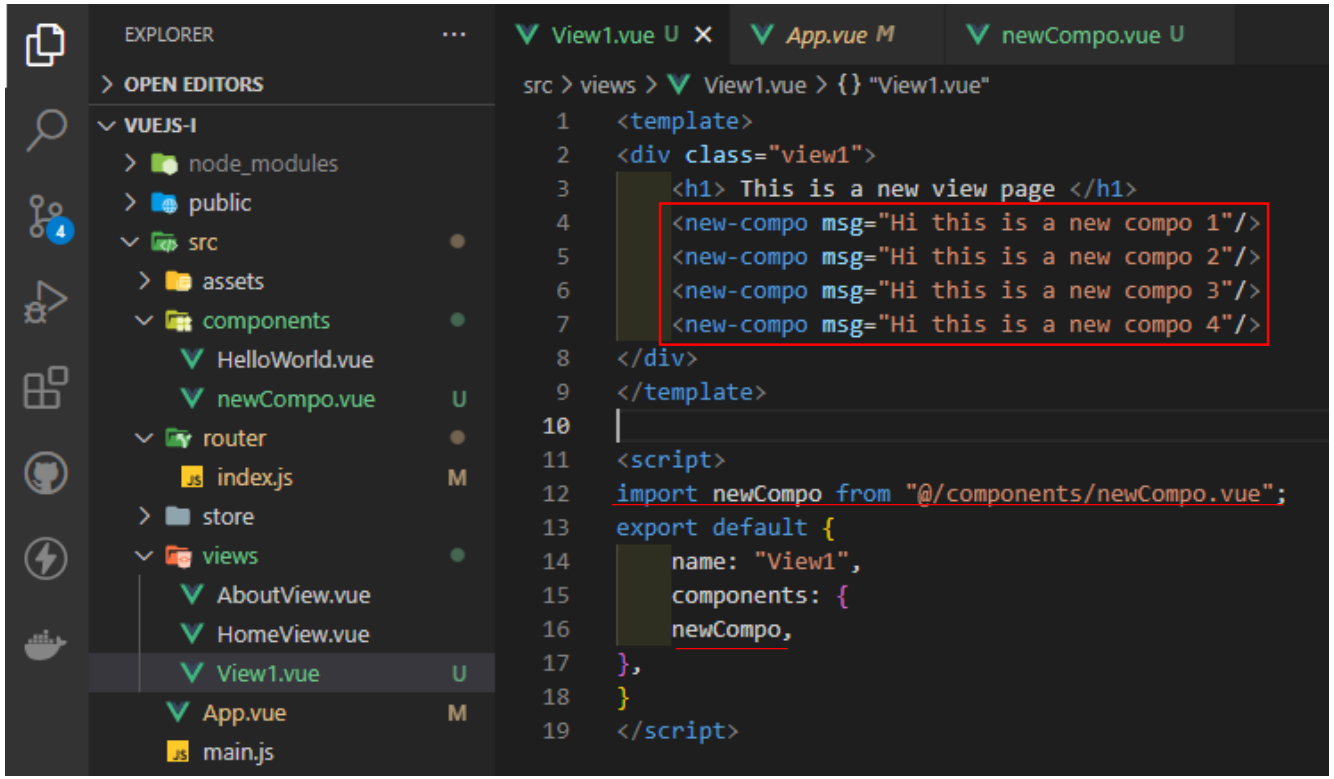
Components

1. **Create a component**
   - In **components** directory, create a new file (e.g., newCompo.vue).
   - Add the structure **<template>**, **<script>,** and **<style>**.
2. Export the **component**, i.e., make it available to be used by other **components** and **views**.

   export default {name: “newCompo", }
3. Register/Import the new component in the using **component/view.**
4. The newly created **component** can be used in another **component** or **view** as **a tag** using the key it is registered under.

   <newCompo />

   <new-compo> </new-compo>



Components

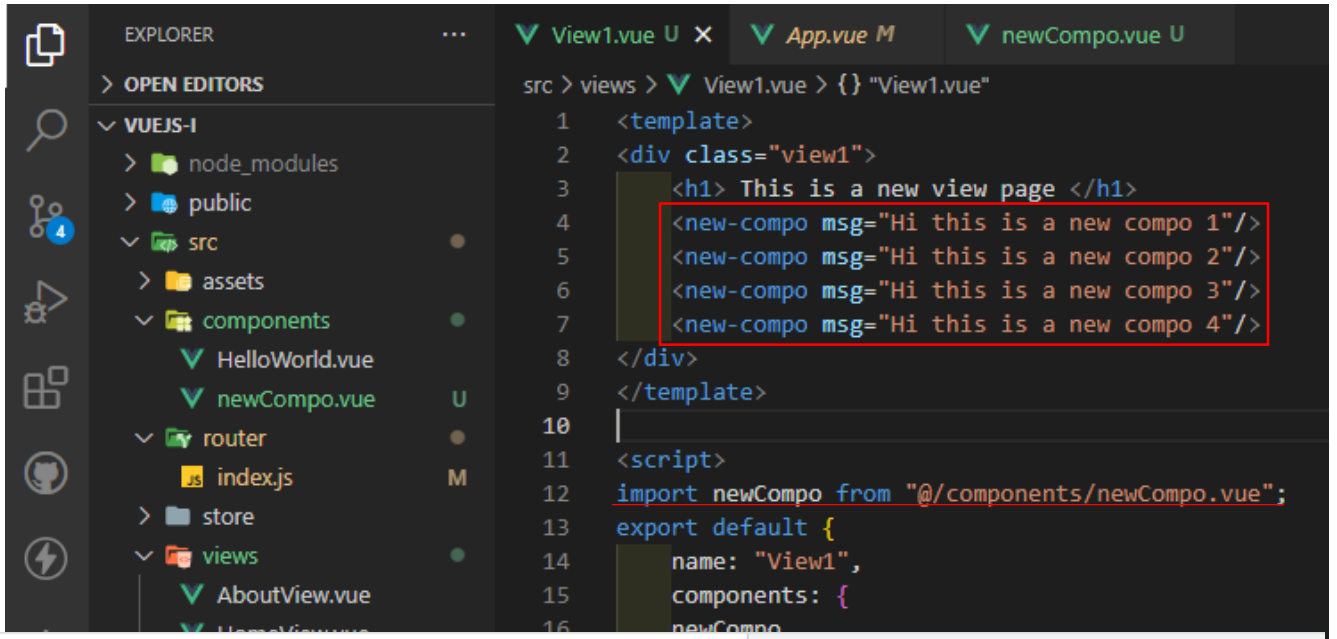
1. **Create a component**
   - In **components** directory, create a new file (e.g., newCompo.vue).
   - Add the structure **<template>**, **<script>,** and **<style>**.
2. Export the **component**, i.e., make it available to be used by other **components** and **views**.

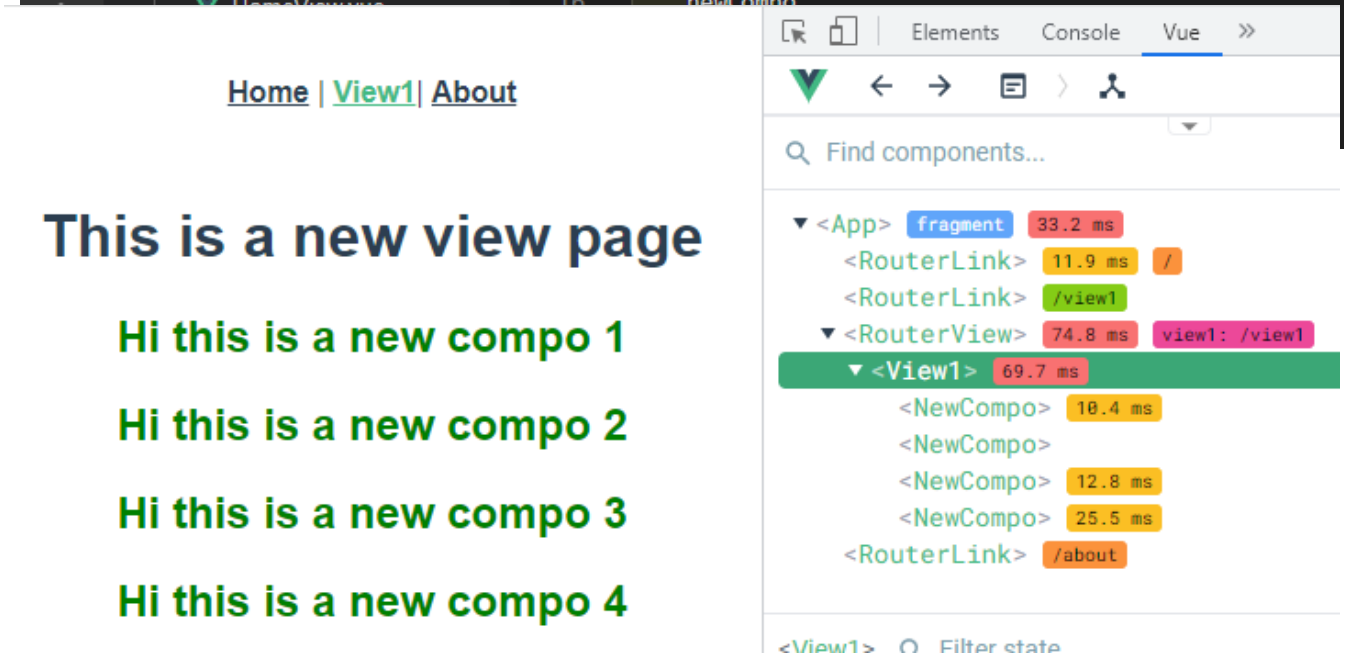
export default {name: “newCompo", }

3. Register/Import the new component in the using **component/view.**
4. The newly created **component** can be used in another **component** or **view** as **a tag** using the key it is registered under.

<newCompo />

<new-compo> </new-compo>





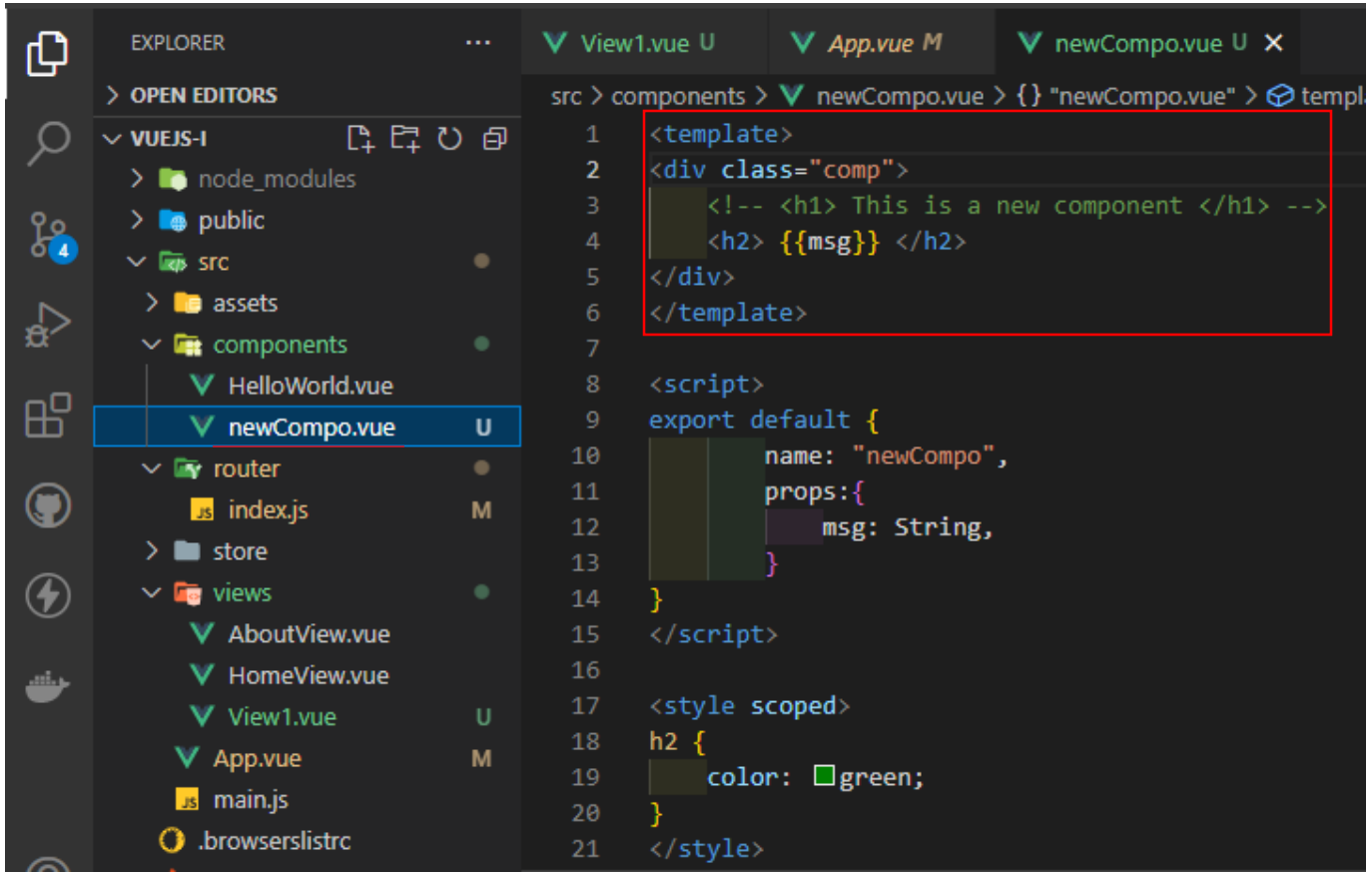
Components - <template>

<template> is an HTML-based template that allows binding the rendered DOM to the underlying component instance's data.

<template> contains valid HTML that can be parsed by browsers and HTML parsers.

To take full advantage of the <template>, we need to understand:

- Data declaration and use (Data type checks and other validators).
- Passing data to components (**props:{}**).
- Computed & Watched properties.
- **Directives.**

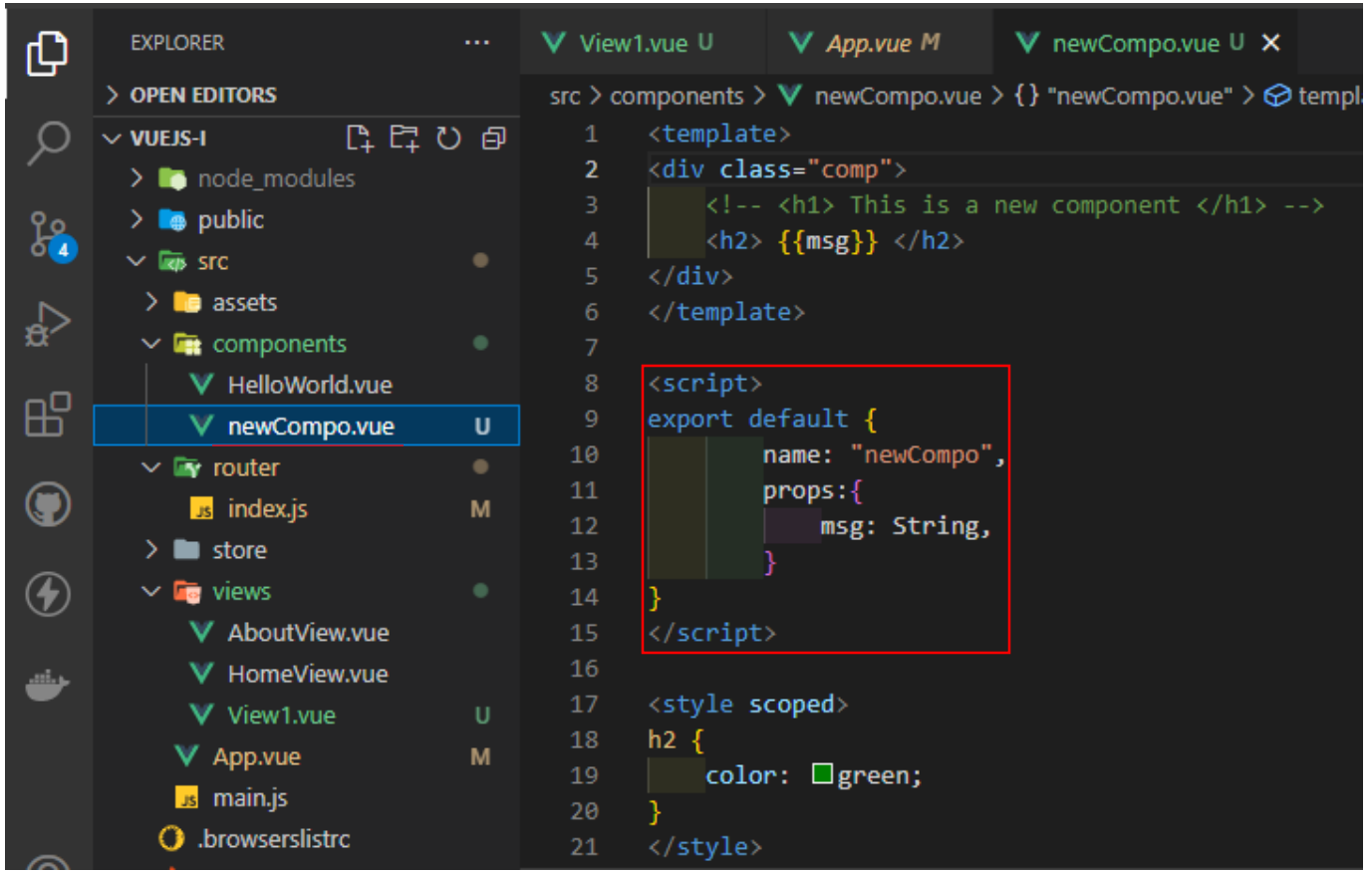


Components - <script>

<script> each *.vue file **may** contain a **<script>** block.

<script> contains the "**data**" and most of the **logic ()** in the component, such as:

- Data declaration (Data type checks and other validators).
- Passing data to components (props:{}).
- Computed & Watched properties
- Methods.
- Lifecycle Hooks.



Components - <style>

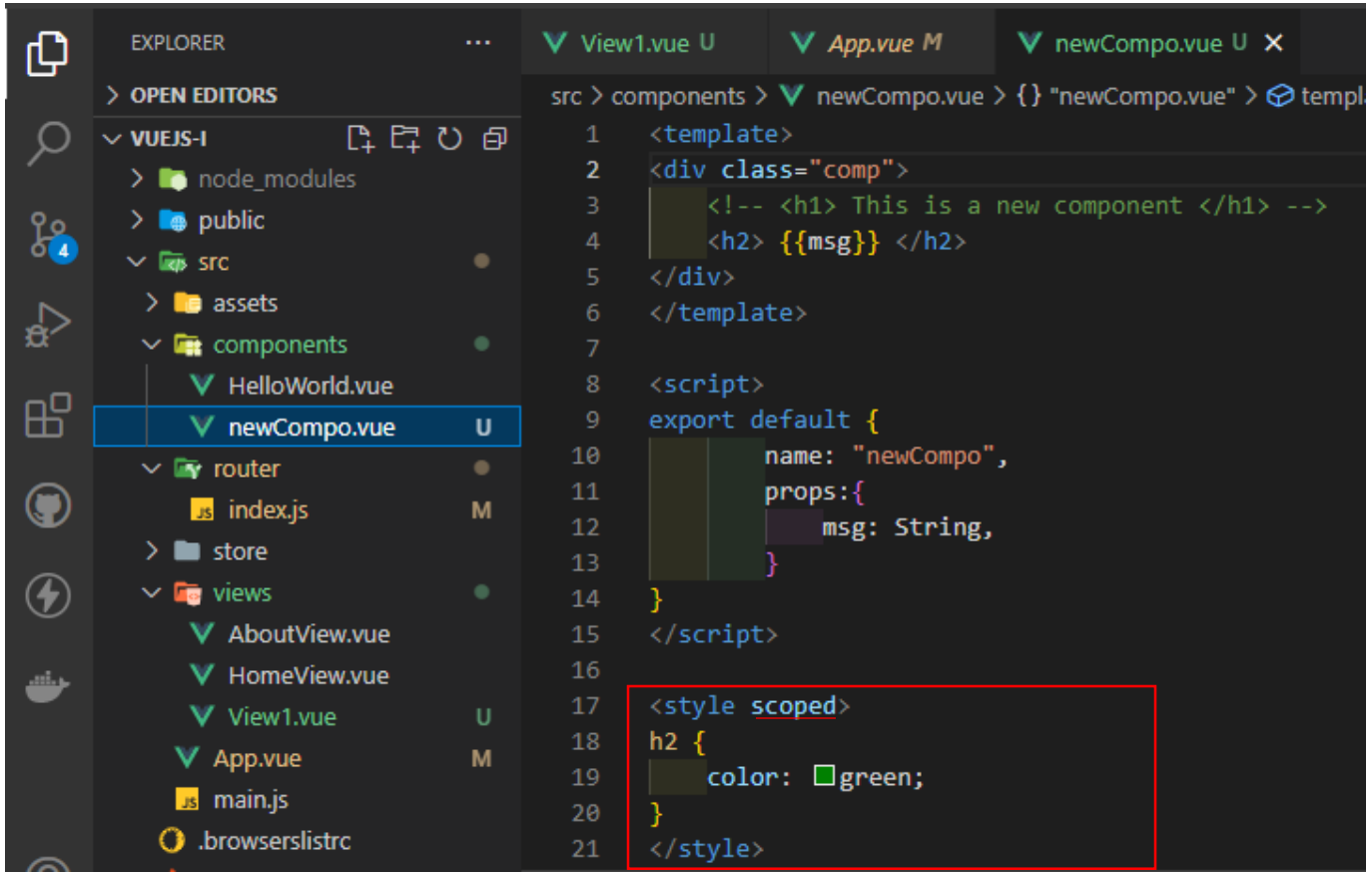
<style> each *.vue file **may** contain a **<style>** block.

<style> contains the “styling” of the component In components, which is written in **CSS.**

CSS is the language that can be use to style an HTML document.

<style scoped> is used to limited the application of the defined styling to the component.

keep an eye on the <style>



Data, Props and Methods

Data declaration and use

Data must be declared as a **function** that returns the data object(s).

Supported data types (String, Number, Boolean, Array, Object, Date)

Why? because if we declared data as an object, it will be shared by reference across all created component instances.

By providing a data function, every time a new instance is created we can call it to return a fresh copy of the initial data.

The most basic form of data binding is text interpolation using the **"Mustache" {{}}**, which can be also used for Using JavaScript Expressions.

src > components > newCompo.vue > {} "newCompo.vue" > script > default

```
1   <template>
2   <div class="comp">
3       <!-- <h1> This is a new component </h1> -->
4     <!--  <h2> {{'Props: ' + msg}} </h2> -->
5       <h3> {{'Data: ' + HiText}} </h3>
6   </div>
7   </template>
8
9   <script>
10  export default {
11      name: "newCompo",
12      props:{
13          msg: String,
14      },
15      data:()=>{
16      return{
17      HiText: "We are learning Vue",
18      isAvailable: Boolean,
19      objArray : [
20      {id: 1, name: 'name1', type: 'type1', description: 'description1'},
21      {id: 2, name: 'name2', type: 'type2', description: 'description2'},
22      {id: 3, name: 'name3', type: 'type3', description: 'description3'},
23      ],
24      }},
25  }
26  </script>
```

Using JavaScript Expressions: https://vuejs.org/guide/essentials/template-syntax.html#using-javascript-expressions

Data declaration and use

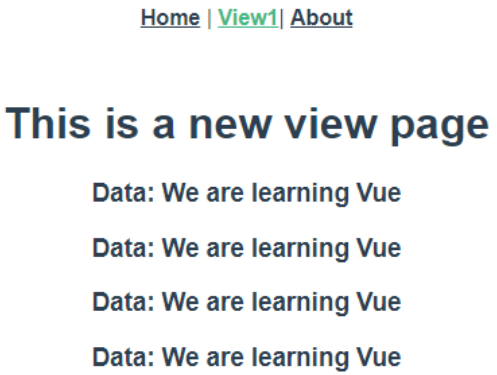
Data must be declared as a **function** that returns the data object(s).

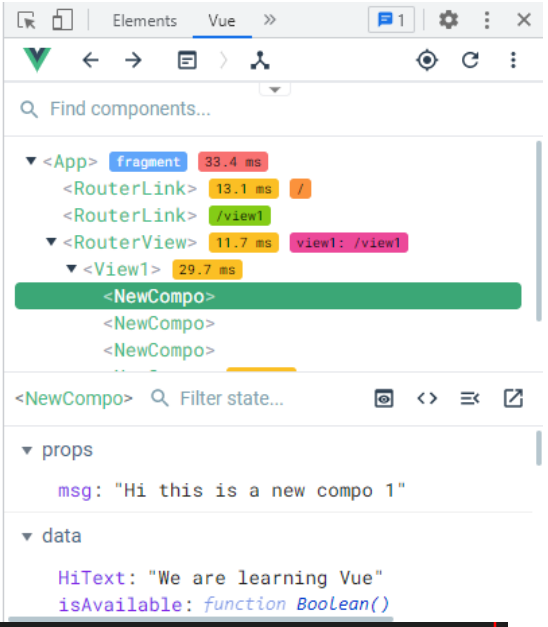
Supported data types (String, Number, Boolean, Array, Object, Date)

Why? because if we declared data as an object, it will be shared by reference across all created component instances.

By providing a data function, every time a new instance is created we can call it to return a fresh copy of the initial data.

The most basic form of data binding is text interpolation using the **"Mustache" {{}}**, which can be also used for Using JavaScript Expressions.






```
15      data:()=>{
16      return{
17      HiText: "We are learning Vue",
18      isAvailable: Boolean,
19      objArray : [
20      {id: 1, name: 'name1', type: 'type1', description: 'description1'},
21      {id: 2, name: 'name2', type: 'type2', description: 'description2'},
22      {id: 3, name: 'name3', type: 'type3', description: 'description3'},
23      ],
24      }},
25  }
26  </script>
```

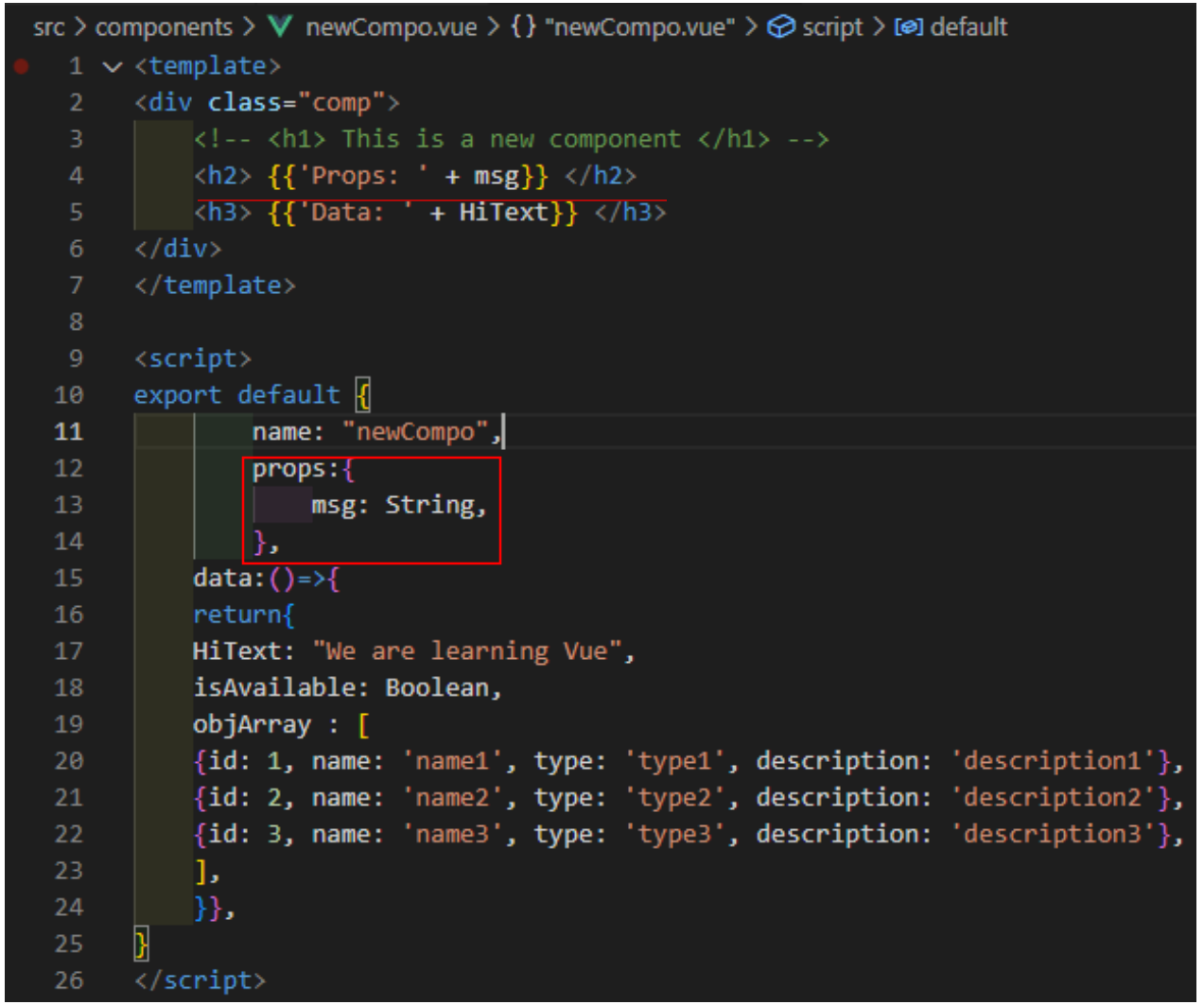
Using JavaScript Expressions: https://vuejs.org/guide/essentials/template-syntax.html#using-javascript-expressions

Props : { }

Props are used to indicate what external data should be passed to the component.

Props vs Data: props are meant to be propagated (passed) and managed from parent components, while data is the component internal state, i.e., the component is responsible for.

Props types: props have types (e.g., String, Number, Boolean, Array, Object, Date, Function).



Props : { }

Props are used to indicate what external data should be passed to the component.

Props vs Data: props are meant to be propagated (passed) and managed from parent components, while data is the component internal state, i.e., the component is responsible for.

Props types: props have types (e.g., String, Number, Boolean, Array, Object, Date, Function).

src > components > newCompo.vue > {} "newCompo.vue" > script > default

```
1  <template>
2    <div class="comp">
3      <!-- <h1> This is a new component </h1> -->
4      <h2> {{'Props: ' + msg}} </h2>
5      <h3> {{'Data: ' + HiText}} </h3>
6    </div>
7    </template>
```

Home | View1| About

This is a new view page

Props: Hi this is a new compo 1

Data: We are learning Vue

Props: Hi this is a new compo 2

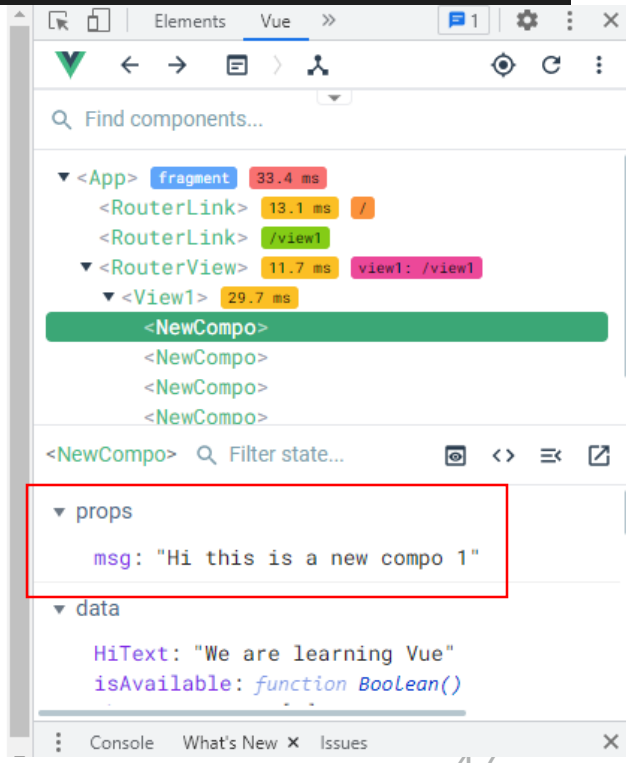
Data: We are learning Vue

Props: Hi this is a new compo 3

Data: We are learning Vue

Props: Hi this is a new compo 4

Data: We are learning Vue



Props checks and other validators

Components can specify requirements for their **props:**

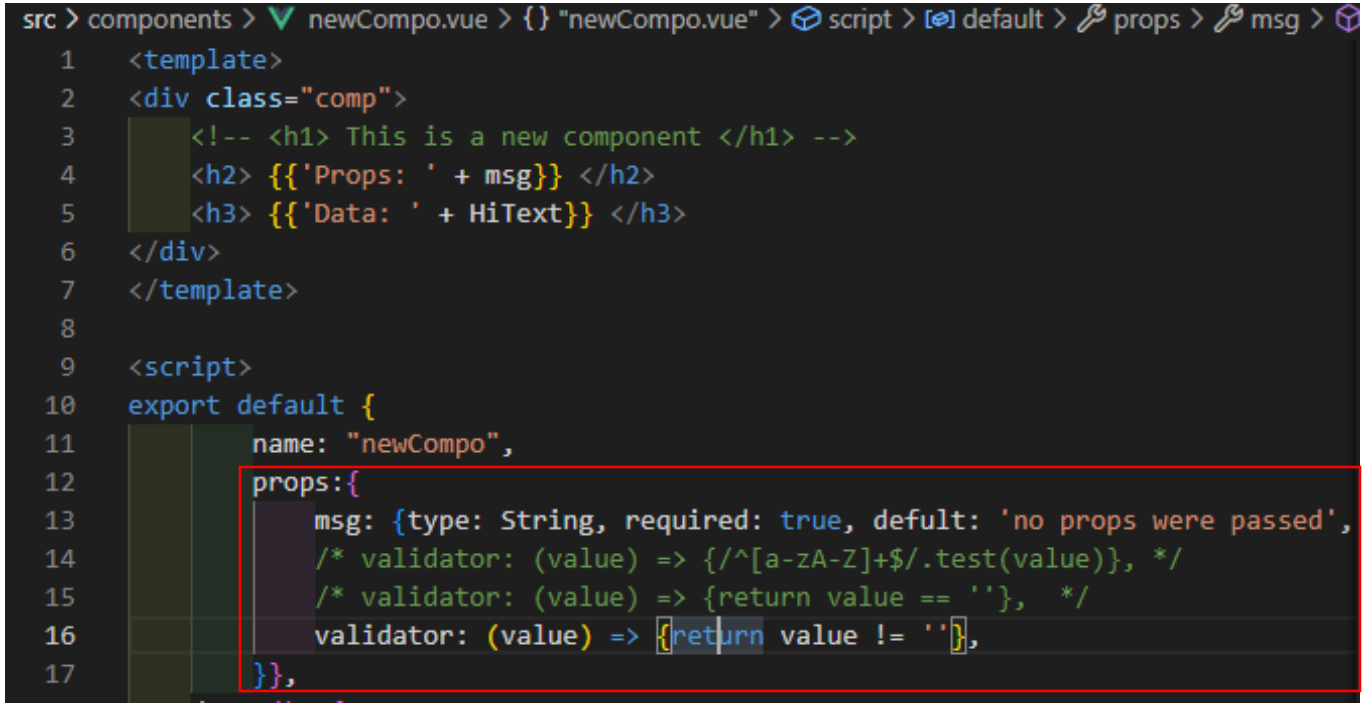
Types checks, Vue checks if a prop has a given type, and will throw a warning if it does not.

Props checks and validators:

Default, If a **default** value is specified, it will be used if the resolved prop value is **undefined**.

Required, All props are optional by default, unless **required: true** is specified.

Custom validator function that takes the prop value as the sole argument.



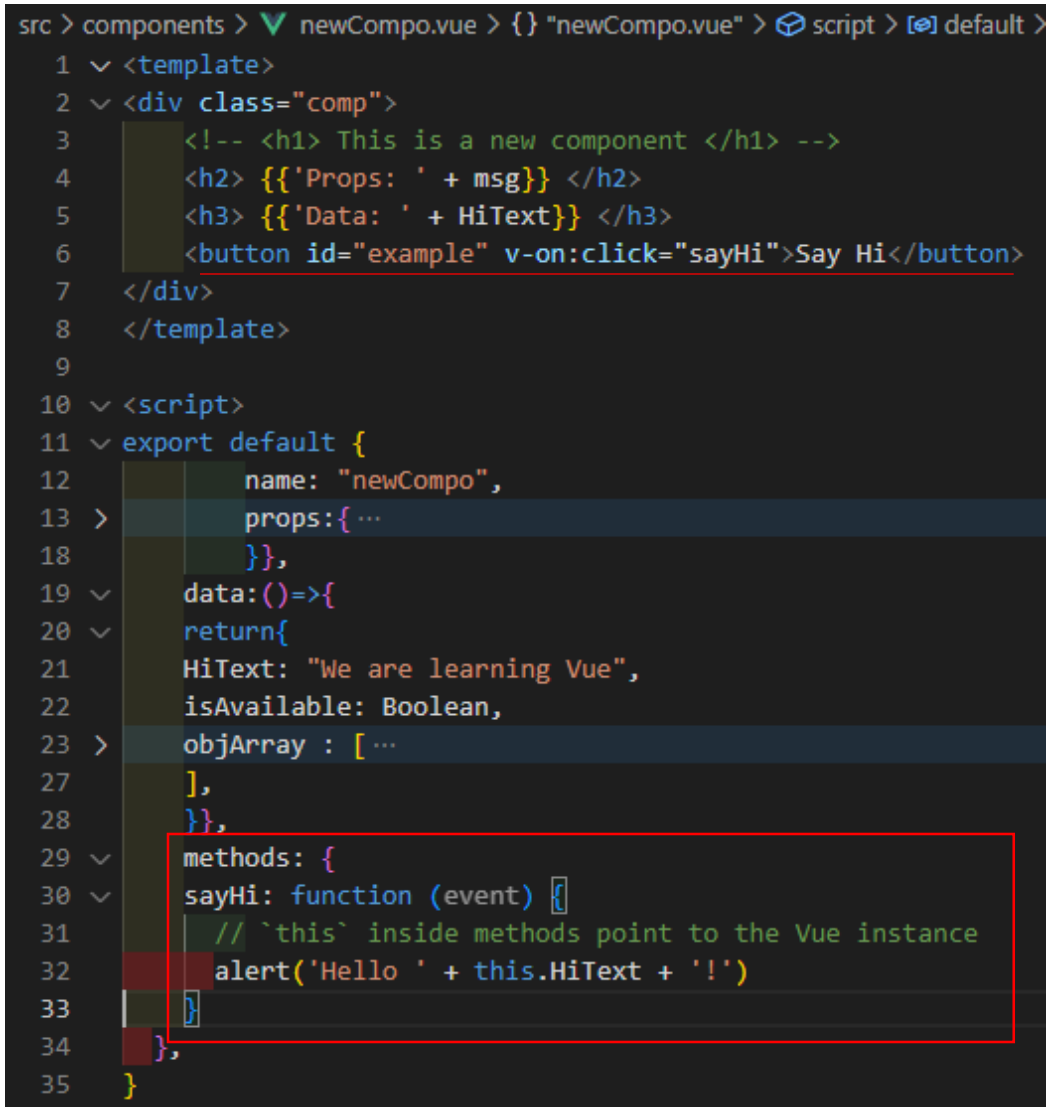
Note: when prop validation **fails**, Vue will produce a console warning.

https://vuejs.org/guide/components/props.html#prop-validation

Methods

Methods can be accessed directly in the Vue instance, or they can be used in directive expressions.

All methods will have their **`this`** context automatically bound to the Vue instance.



Directives

A directive is a special token in the markup that allows attaching a specified behaviour to that DOM elements

https://vuejs.org/api/built-in-directives.html

Directives: v-text & v-html

v-text: updates the element's text content, so it will overwrite any existing content inside the element.

```
<span v-text="msg"></span>
```

v-text vs. {{}}, if you need to update a part of text content, it is better to **{{}}**.

v-html: updates the element's innerHTML, and it expects String.

```
<p v-html="messageHtml"></p>
<!-- messageHtml: "<h1> Hi I am an HTML<h1/>“ -->
```

```
<!-- <v-text> -->
    <p v-text= "message"> </p>
    |
<!-- <v-html> -->
    <p v-html= 'messageHtml'> </p>
```

```
data: function() {
  return {
  message: "Hi, I am a v-text",
  messageHtml: "<h1> Hi, I am a v-HTML<h1/>",
  }
}
```

Hi, I am a v-text

Hi, I am a v-HTML

Directives: v-on

v-on (Shorthand: “@”) Attaches an event listener to an element. The event type is denoted by the argument. The expression can be a method or an inline statement.

```
<!-- method handler -->
<button v-on:click="dosomething"></button>

<!-- inline statement -->
<button v-on:click=" dosomething('hello', $event)"></button>
```

```
<!-- v-on -->
<p @click= 'count++'> Hello {{nameClick}}, you have clicked {{count}} times </p>
```

```
data: function() {
return {
nameClick: 'John',
count: 0,
}
```

Hello John, you have clicked 7 times

Directives: v-show & v-once

v-show: toggle the element's visibility based on the truthfulness of the expression value.

v-show works by setting the display CSS property via inline styles, and will try to respect the initial display value when the element is visible. It also triggers transitions when its condition changes.

<h1 **v-show**="true"> Hello! </h1>

<h1 **v-show** ="count>4"> count: {{count}}</h1>

v-once: render the element and component once only, and skip future updates.

On subsequent re-renders, the element/component and all its children will be treated as static content and skipped.

<new-compo **v-once** msg = "This is a new compo" />

```
<!-- <v-show> -->
<h1 v-show="true">Hello!</h1>
<h1 v-show ="count>4"> v-show test count: {{count}}</h1>
```

v-show test count: 7

```
<!-- v-once -->
<h2> v-once through componenet </h2>
<new-compo v-once msg="Hi this is a new compo - v-once"/>
```

v-once through componenet

Props: Hi this is a new compo - v-once

Data: We are learning Vue

Say Hi

Directives: v-if, v-else-if & v-else

v-if: conditionally render an element or a template fragment based on the truthy-ness of the expression value.

<p **v-if**="Math.random() > 0.4"> Appear if random number bigger than 0.4 </p>

v-else-if - Restriction: previous sibling element must have v-if or v-else-if.

<p **v-else-if**="Math.random() == 0.4"> Appear if random number equal to 0.4 </p>

v-else: denote the "else block" for **v-if** or a **v-if / v-else-if** chain.

<p **v-else**> "random number not bigger than 0.4" </p>

```
<p v-if= "Math.random() > 0.4"> Appear if random number bigger than 0.4  </p>
<p v-else-if = "Math.random() == 0.4"> Appear if random number equal  to 0.4  </p>
<p v-else> "random number not bigger than 0.4" </p>
```

Directives: v-bind & v-model

v-bind (Shorthand “:”): dynamically bind one or more **attributes**, or a **component prop** to an expression.

```
<!-- bind an attribute -->
<img v-bind:src="imageSrc">
<!-- shorthand -->
<input type = "text" :value="writeSomething“>
```

v-model: create a two-way binding on a form input element or a component.

```
<input type = "text" v-model ="writeSomething“>
```

Hard coded Image source

Vue logo

Image source provided without bind

Image source provided with bind

using v-bind	using v-model

You wrote

Directives: v-for

v-for: render the element or template block multiple times based on the source data.

View: view1.vue

```
<como-list= :compoArray="compoArray"/>
..
data: function() {
  return {
  compoArray : [
    {id: 1, msg: 'msg description 1'},..}
```

Component: compoArray.vue

```
<ul>
<li v-for= "compo in compoArray" :key="compo.id">
<span> <b>Id:</b>{{compo.id}} </span><br/>
<span> <b> Msg desc.:</b> {{compo.msg}} </span>
</li>
</ul>
```

```
<!-- v-for -->
<h2> v-for, props: and components </h2>
<compo-list :compoArray="compoArray"/>
```

```
compoArray : [
    {id: 1, msg: 'msg description 1'},
    {id: 2, msg: 'msg description 2'},
    {id: 3, msg: 'msg description 3'},
    ],
}
```

src > components > compoList.vue > {} "compoList.vue" > template > div#comp-list >

```
1  <template>
2  <div id= "comp-list">
3  <h1>This is a list of Componenet </h1>
4  <ul>
5  <li class="item" v-for= "compo in compoArray" :key="compo.id">
6  <span class="id"> <b>Id:</b> {{compo.id}} </span><br/>
7  <span class="msg"> <b>Msg description:</b> {{compo.msg}} </span>
8  </li>
9  </ul>
10 </div>
11 </template>
12
13 <script>
14 export default {
15   name: "compoList",
16 props:   ["compoArray"],
17 data:()=>{
18     return{
19   }},
20  };
21 </script>
```

Directives: v-for

v-for: render the element or template block multiple times based on the source data.

View: view1.vue

```
<como-list= :compoArray="compoArray"/>
..
data: function() {
  return {
  compoArray : [
    {id: 1, msg: 'msg description 1'},..}
```

Component: compoArray.vue

```
<ul>
<li v-for= "compo in compoArray" :key="compo.id">
<span> <b>Id:</b>{{compo.id}} </span><br/>
<span> <b> Msg desc.:</b> {{compo.msg}} </span>
</li>
</ul>
```

```
<!-- v-for -->
<h2> v-for, props: and components </h2>
<compo-list :compoArray="compoArray"/>
```

```
compoArray : [
    {id: 1, msg: 'msg description 1'},
    {id: 2, msg: 'msg description 2'},
    {id: 3, msg: 'msg description 3'},
    ],
}
```

src > components > compoList.vue > {} "compoList.vue" > template > div#comp-list >

```
1  <template>
2  <div id= "comp-list">
3  <h1>This is a list of Componenet </h1>
4  <ul>
5  <li class="item" v-for= "compo in compoArray" :key="compo.id">
6  <span class="id"> <b>Id:</b> {{compo.id}} </span><br/>
7  <span class="msg
8  </li>
9  </ul>
10 </div>
11 </template>
12
13 <script>
14 export default {
15   name: "compoLi
16 props:   ["compo
17 data:()=>{
18     return{
19   }},
20  };
21 </script>
```

Computed and watched properties

https://vuejs.org/guide/essentials/computed.html
https://vuejs.org/guide/essentials/watchers.html

Computed properties

computed properties enable you to create a property that can be used to modify, manipulate, and display data within your components in a readable and efficient manner.

computed properties is a way to reduce the complexity of the <template> as it can become bloated and hard to maintain.

Unlike **methods**, **computed properties** will be “re-calculated” anytime some data changes in the component.

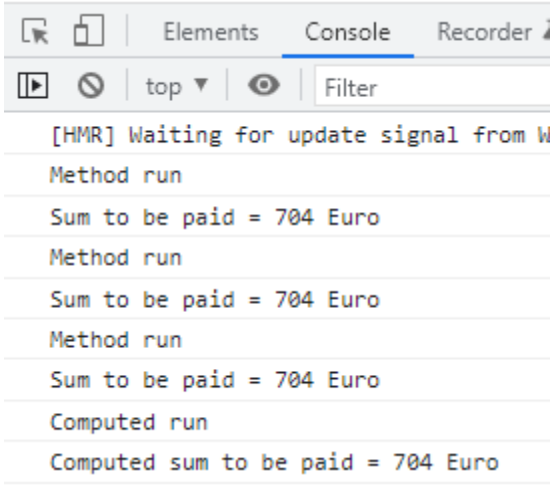
Computed properties

computed properties enable you to create a property that can be used to modify, manipulate, and display data within your components in a readable and efficient manner.

computed properties is a way to reduce the complexity of the <template> as it can become bloated and hard to maintain.

Unlike **methods**, **computed properties** will be “re-calculated” anytime some data changes in the component.

```
<p> {{toBePaid()}} </p>
<p> {{toBePaid()}} </p>
<p> {{toBePaid()}} </p>

<p> {{toBePaidComputed}} </p>
<p> {{toBePaidComputed}} </p>
<p> {{toBePaidComputed}} </p>
```




```
methods: {
    toBePaid: function () {
        console.log("Method run");
        var Methodmoney =  "Sum to be paid = " + (this.billsTobepaid[0].price + this.billsTobepaid[1].price) + " Euro";
        console.log(Methodmoney)
        return Methodmoney;
    }
  },

computed:{
    toBePaidComputed: function () {
  console.log("Computed run");
  var money = "Computed sum to be paid = " + (this.billsTobepaid[0].price + this.billsTobepaid[1].price) + " Euro";
  console.log(money)
  return money;
    }
  }
```

Watched properties (Watchers)

Watchers allow the developers to listen to the component data and run whenever they change a particular property.

Watcher is a unique Vue.js feature that lets you keep an eye on one property of the component state and run a function when that property value changes

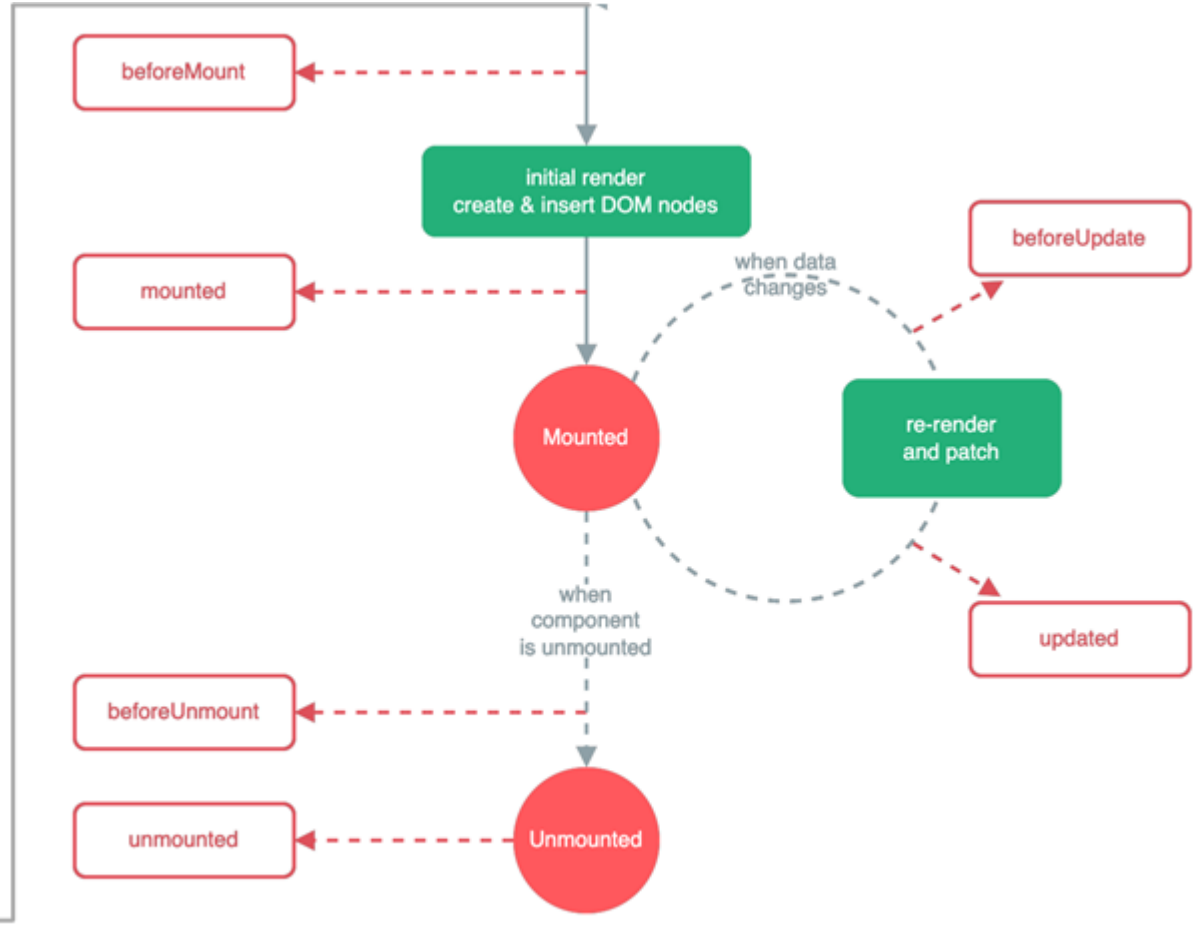
```
<!-- Watched properties (Watchers) -->
<h2> Watched properties (Watchers) </h2>

<label> Pay in Euro </label>
<input type="text" v-model="Euro"><br>

<label> Pay in US Dollar </label>
<input type="text" v-model="Dollar"><br>
```

```
watch:{
Euro: function(w){
    this.Dollar = w *2;
},
Dollar: function(w){
    this.Euro = w /2;
},
}
```

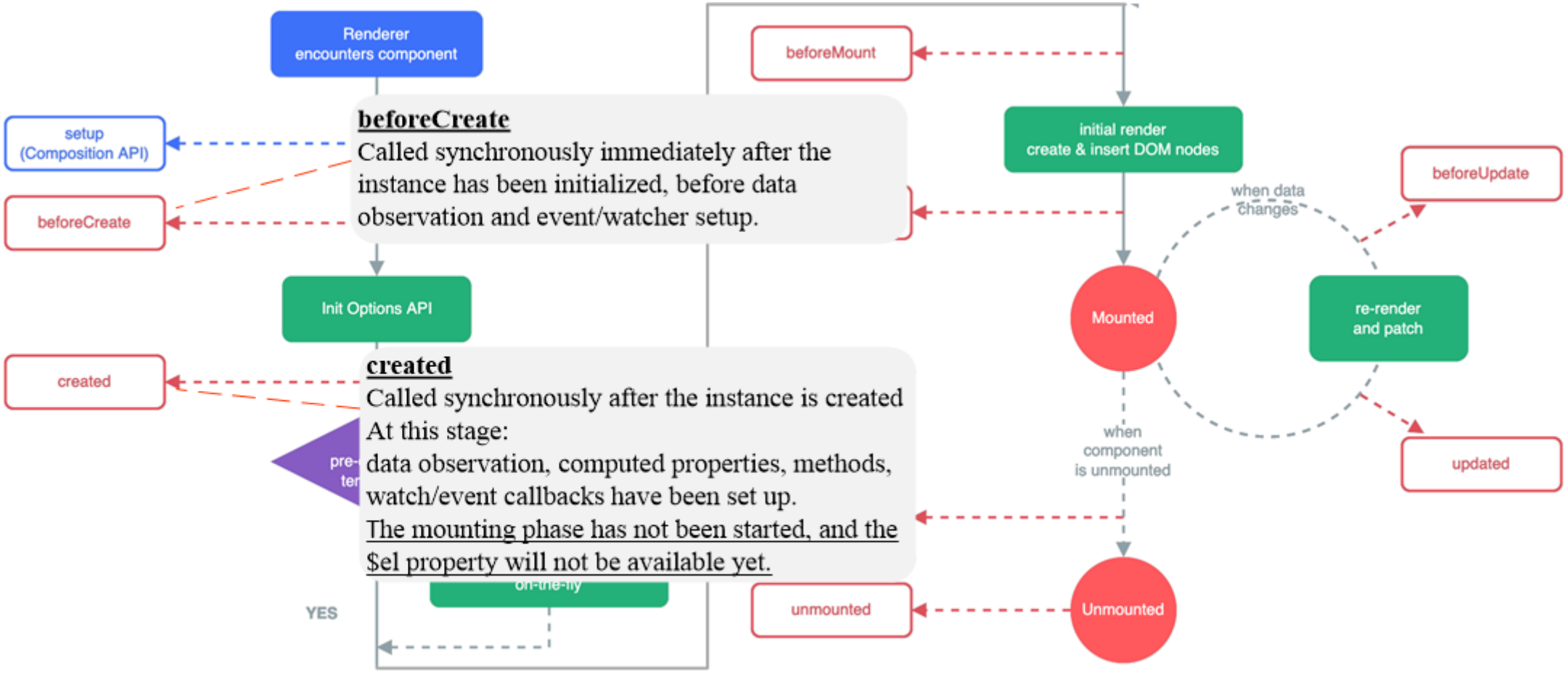
Lifecycle hooks

https://vuejs.org/api/options-lifecycle.html

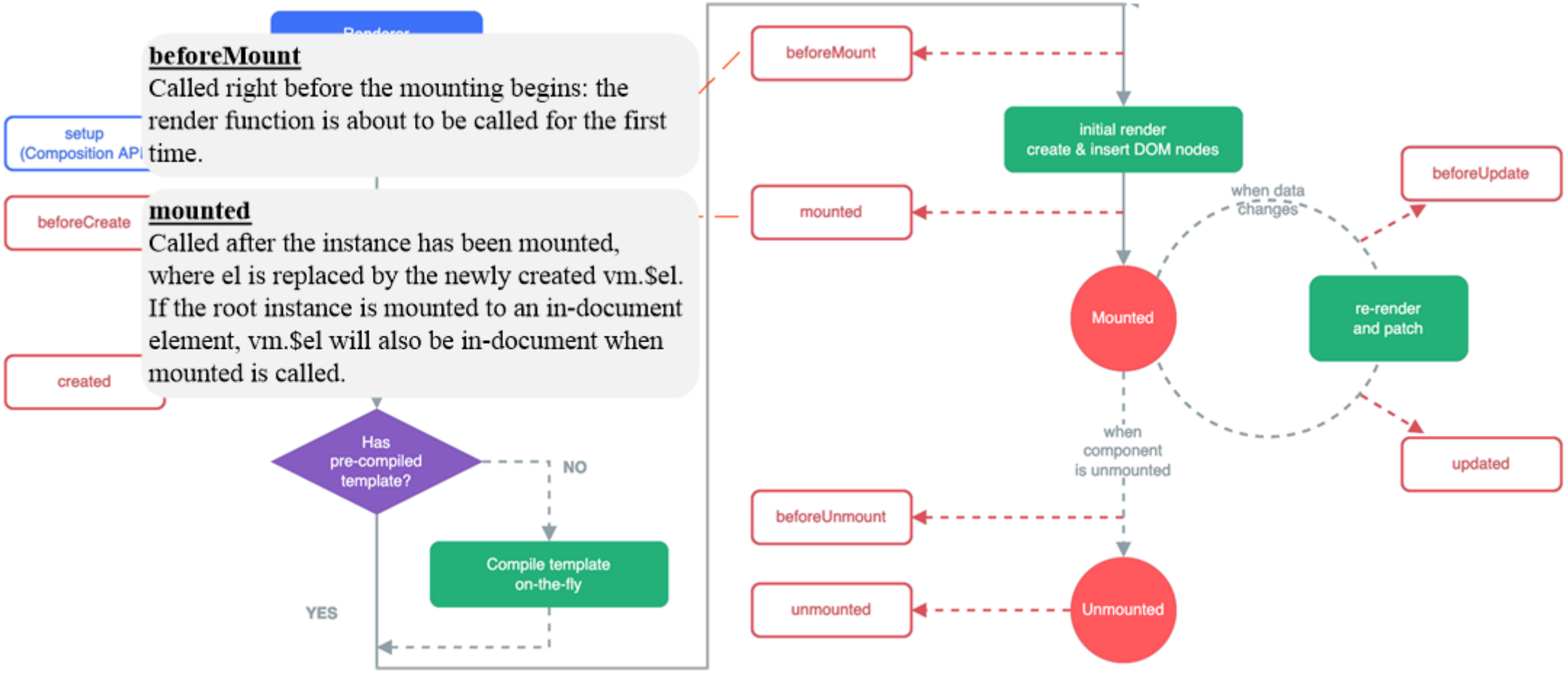
Lifecycle hooks



Lifecycle hooks

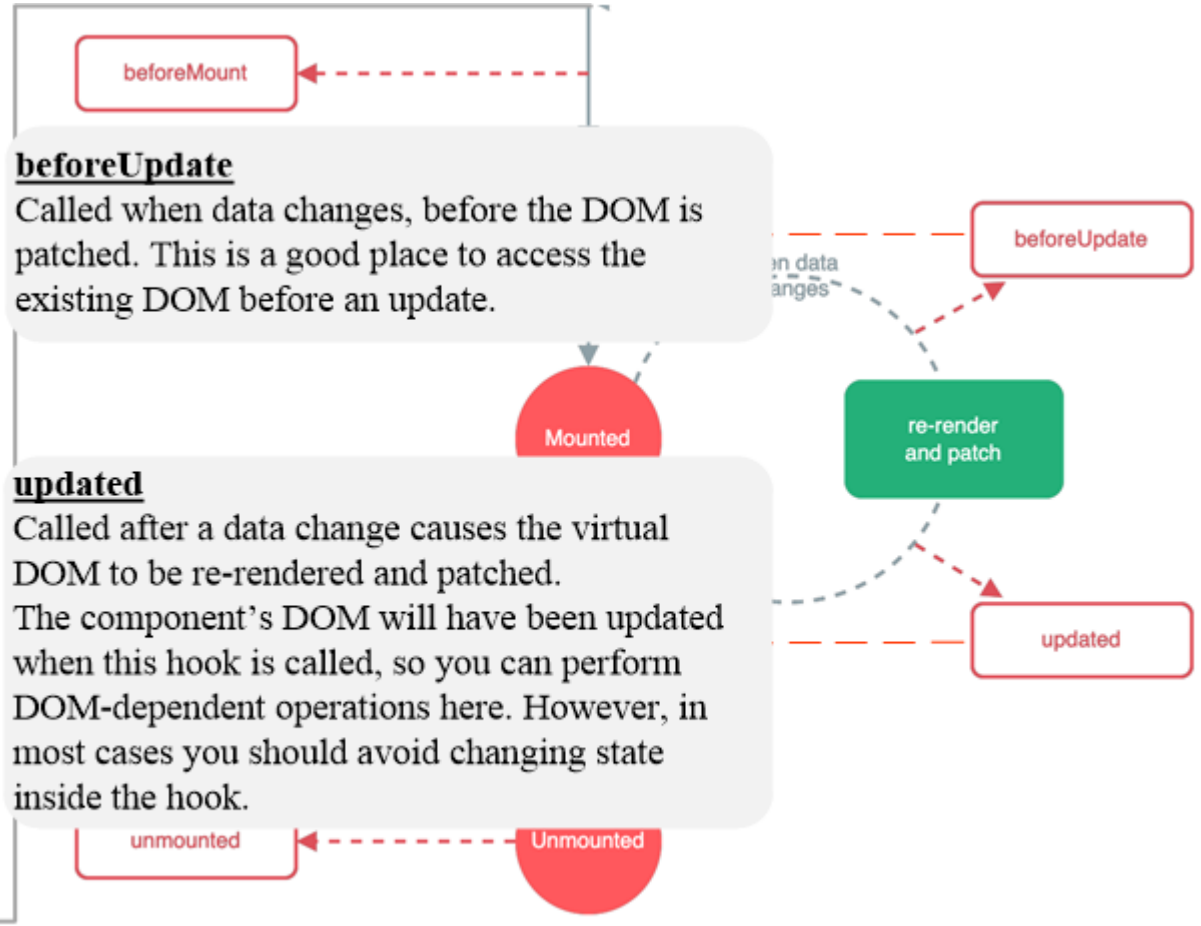


Lifecycle hooks

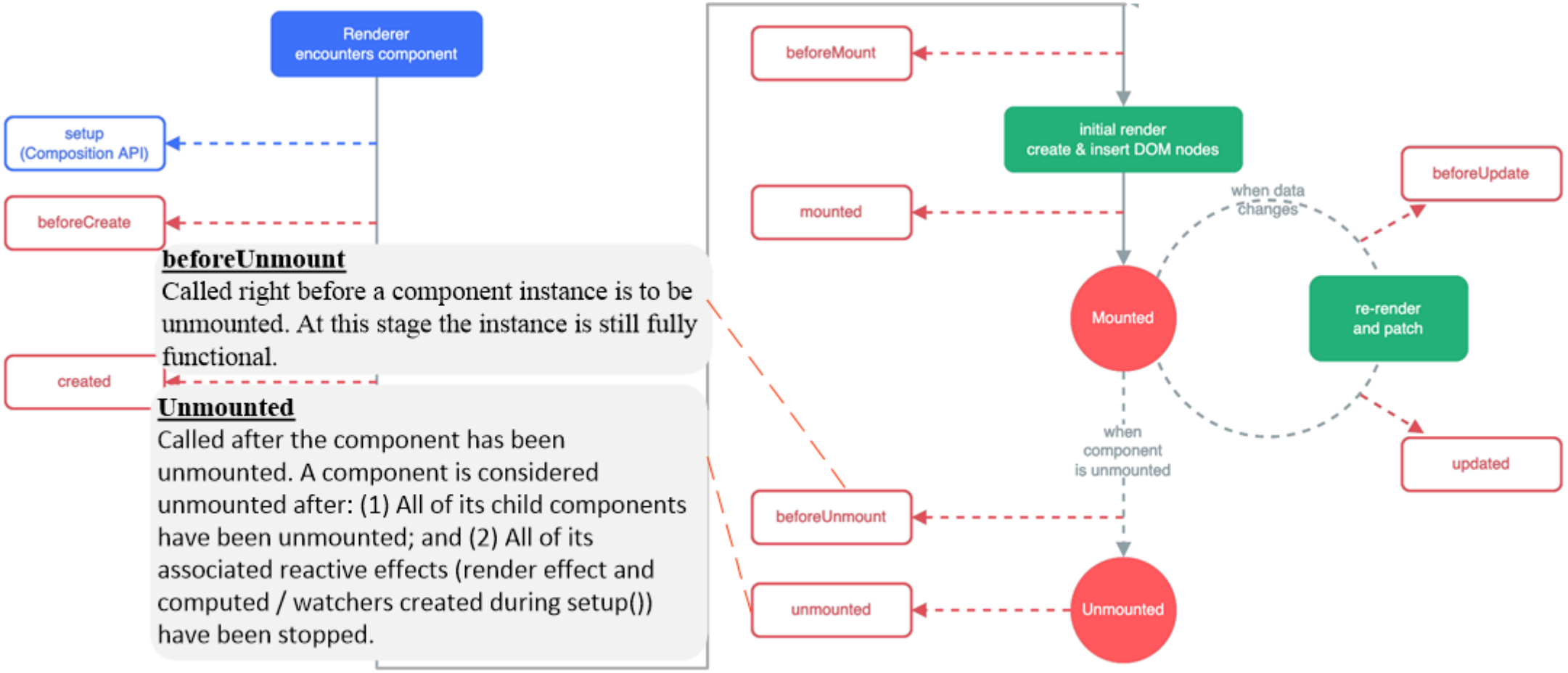


mounted() is commonly used for fetching data from “external sources” to be used in mounted elements.

Lifecycle hooks



Lifecycle hooks



NPM

Node Package Manager

NPM

***npm*[1]** is a package manager for the JavaScript programming language.

npm has its own command-line interface (CLI).

npm offers a registry that is an online database of public and paid-for private package.

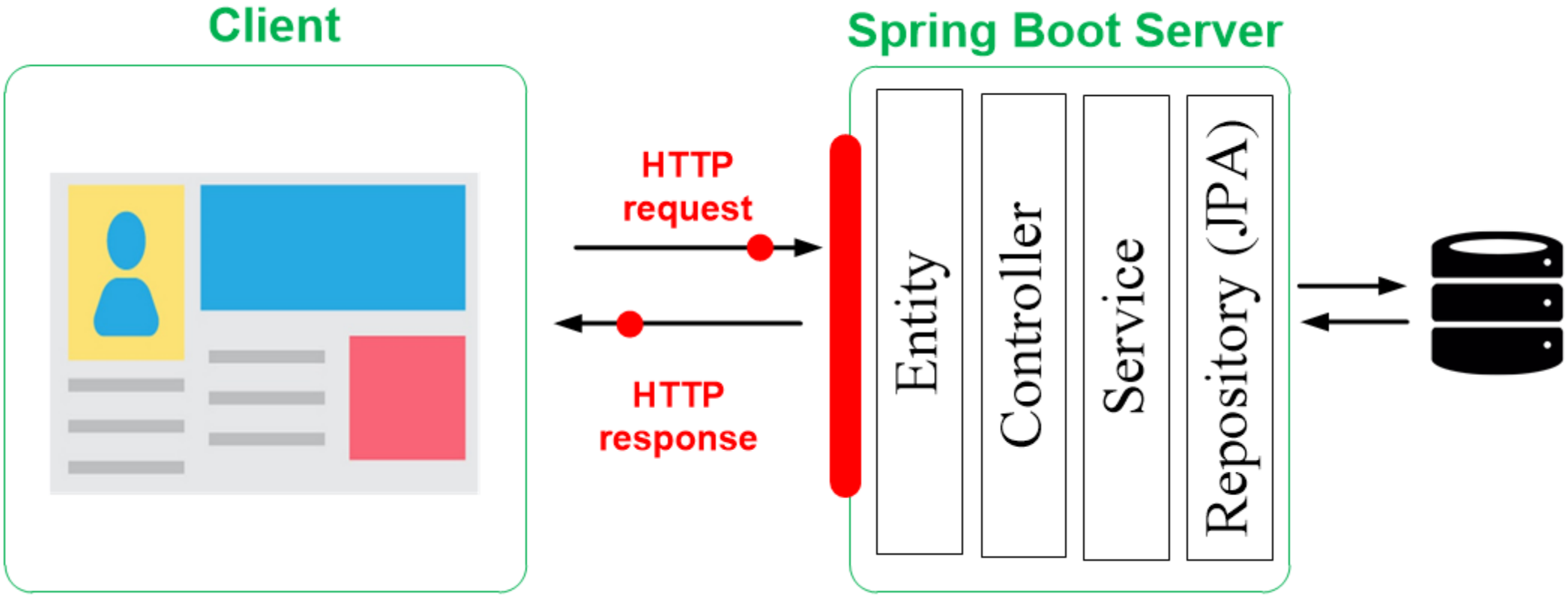
Why ***npm*** is very important: it allows adapting packages of code for your apps, or incorporating packages as they are.

[1]https://www.npmjs.com/

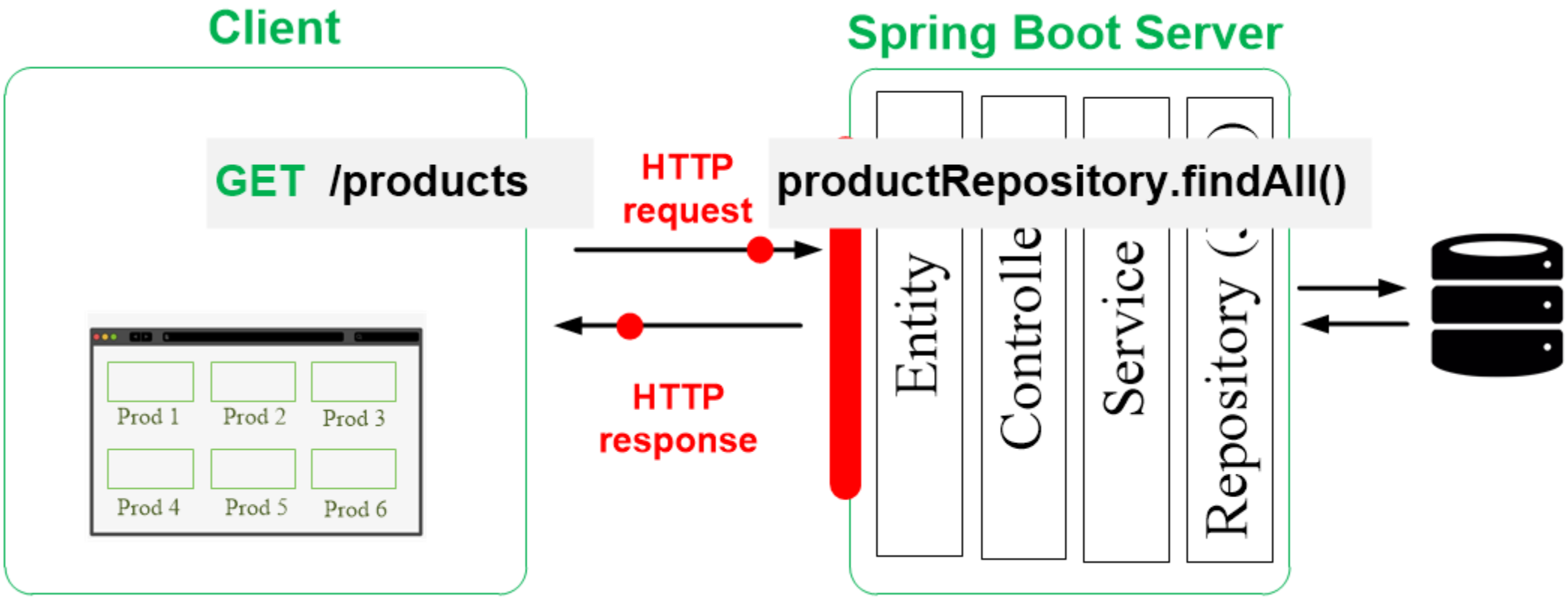
Vue.Js

A practical example

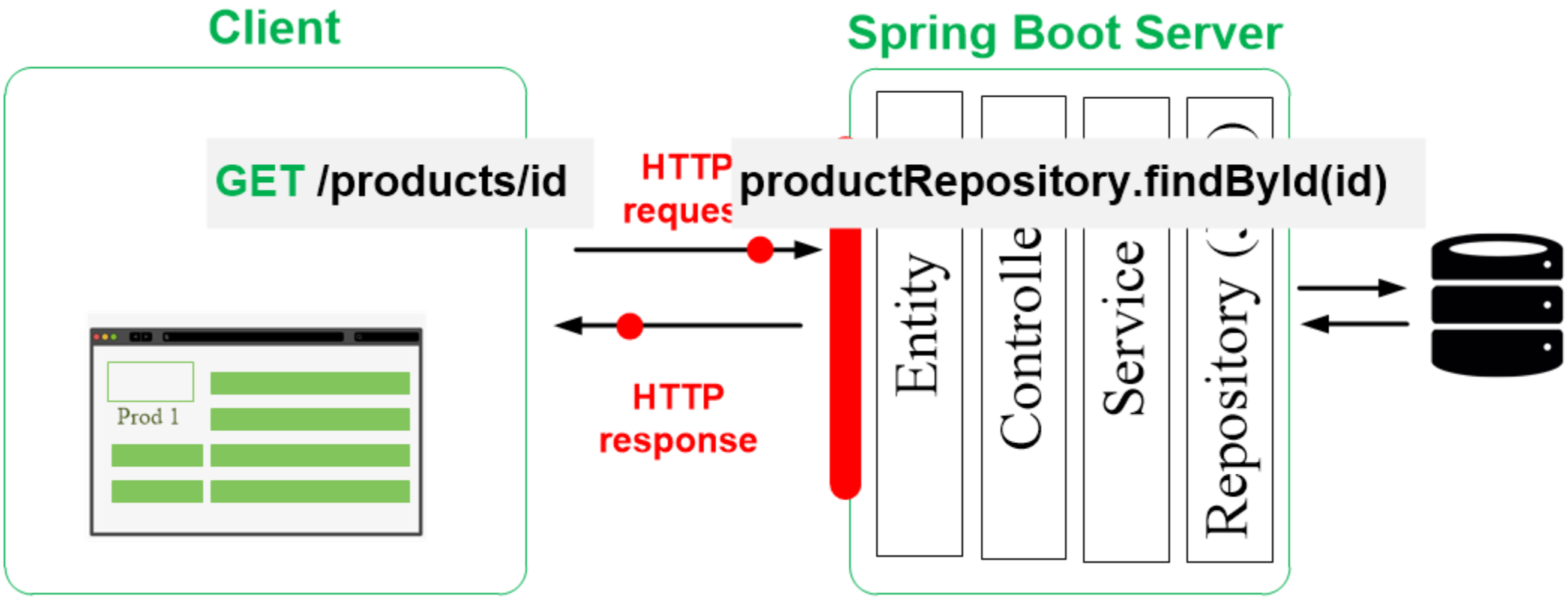
The Client side



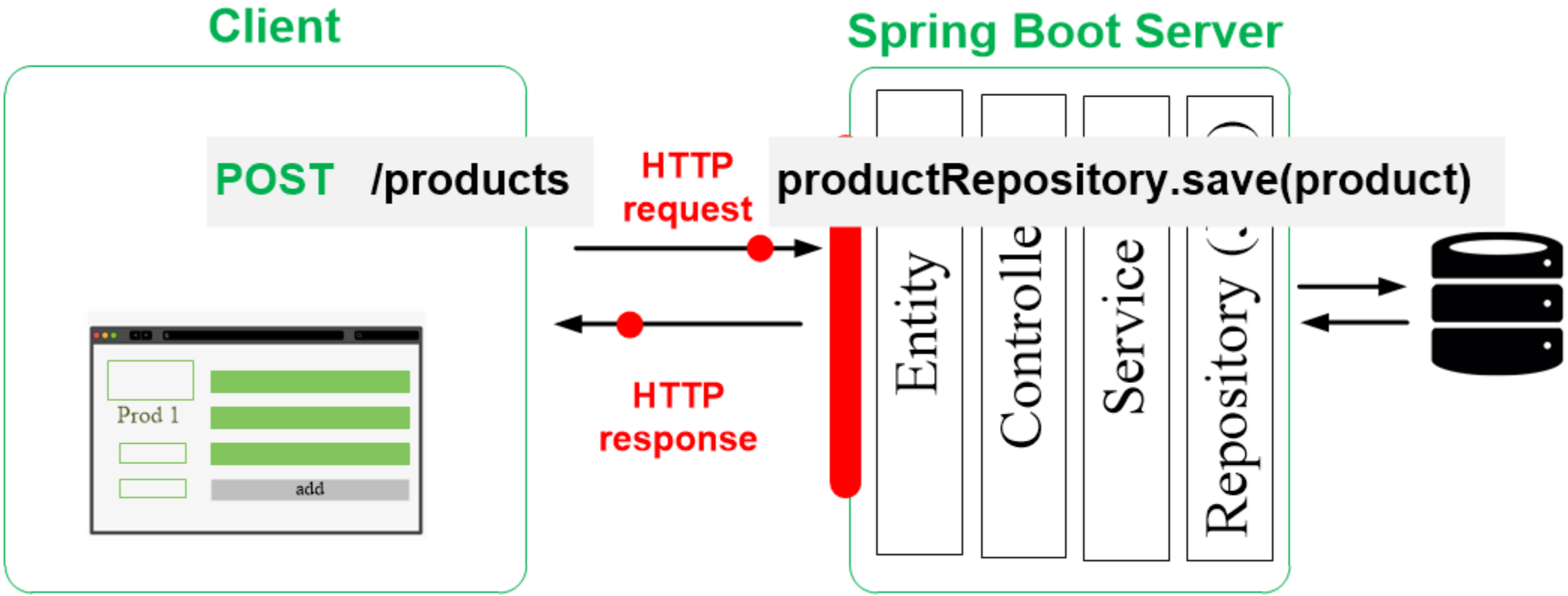
The Client side



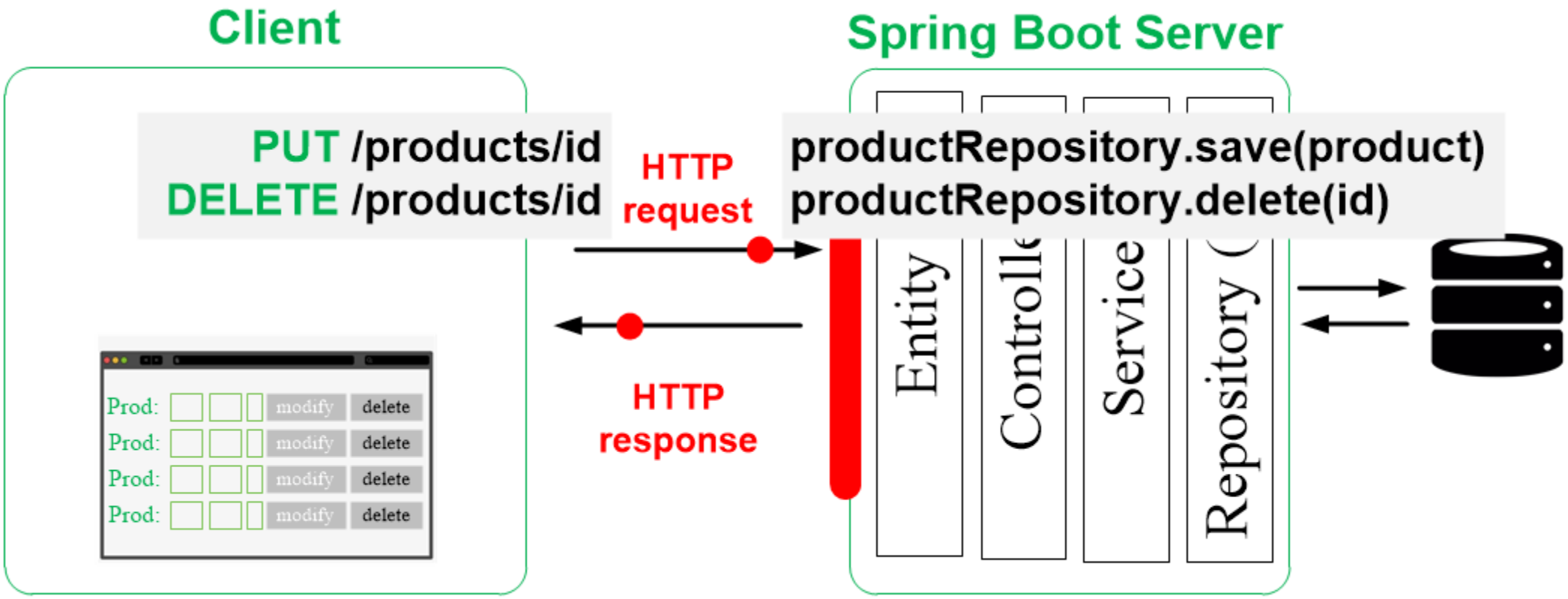
The Client side



The Client side

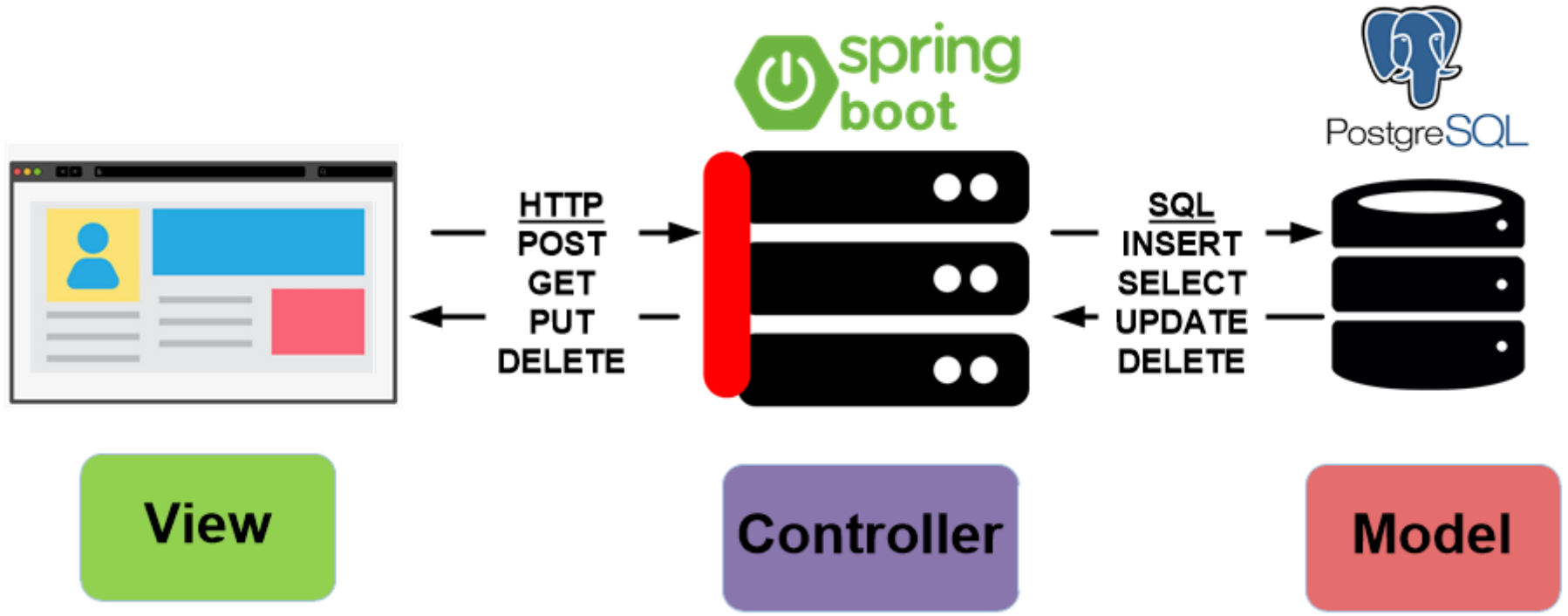


The Client side

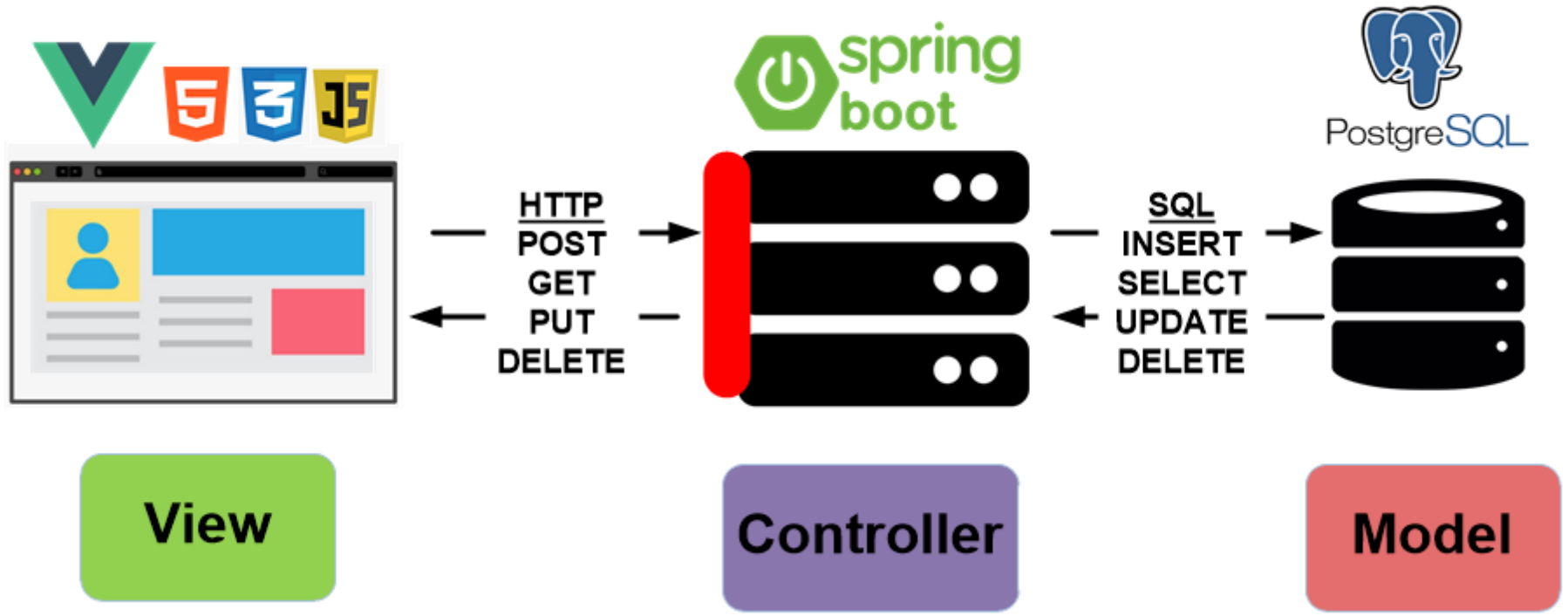


Building an MVC application

Building an MVC application



Building an MVC application



Building an MVC application

We will **develop a simple App** that includes:

The View - a **front-end** (**Vue.js App**) allows to:

- **Create** a product (content) in the database;
- **Fetch** and present all products from the database;
- **Fetch** and present a specific product from the database;
- **Update** a specific product in the database; and
- **Deletes** a specific product from the database.

The Controller - a **backend** (**SpringBoot App**) allows for handling the previous requests.

The Model - a **database** (Postgres) that contain the data of our App.

Building an MVC application

We will **develop a simple App** that includes:

The View - a **front-end** (**Vue.js App**) allows to:

- **Create** a product (content) in the database;
- **Fetch** and present all products from the database;
- **Fetch** and present a specific product from the database;
- **Update** a specific product in the database; and
- **Deletes** a specific product from the database.

The Controller - a **backend** (**SpringBoot App**) allows for handling the previous requests.

The Model - a **database** (Postgres) that contain the data of our App.

Building an MVC application- the Controller

Method	URI/PATH	Actions
POST	/api/products	Creates a new product
GET	/api/products	Fetches all products
GET	/api/products/:id	Fetches a product based on its id
PUT	/api/products/:id	Updates a product based on its id
DELETE	/api/products/:id	Deletes a product based on its id

Building an MVC application- the Controller

Solve the CORS issue

If you are running both the front-end and back-end applications on the same computer, they will be using different ports. This cause a CORS (Cross-Origin Resource Sharing) error.

To solve this issue, you need to use **@CrossOrigin** tag to configure allowed origins.

It should be used before each @RestController

```
…
//@CrossOrigin(origins                                    =
        "http://localhost:8082")
@CrossOrigin(origins = “*")
@RestController
…
```

The View

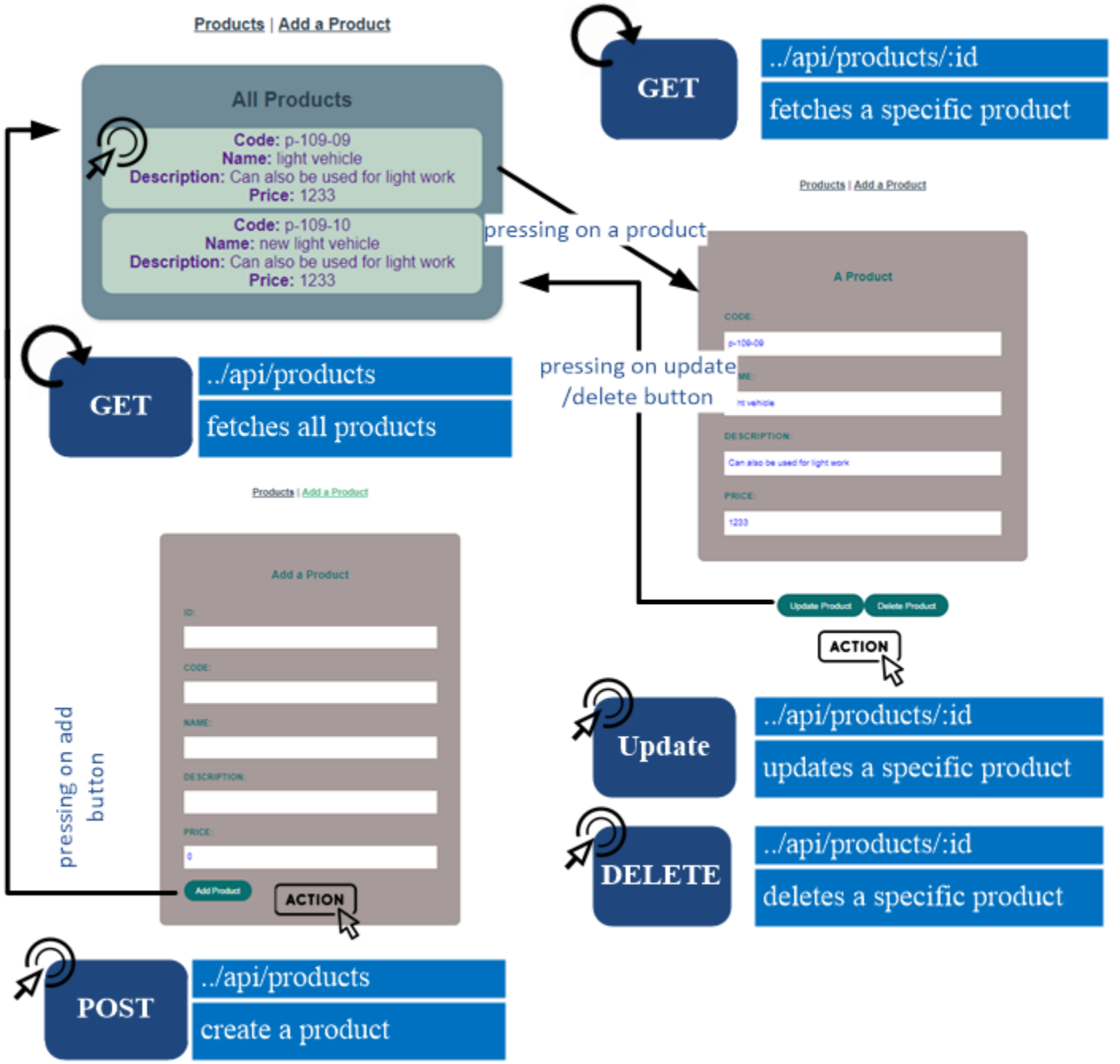
Three **views** are required:

1. **All products:** fetches and presents all products. Pressing on any product, will direct us to the **single product view with the id of the product** as a route parameter**.**
2. **A single product:** fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view.**
3. **A create product:** allows creating a new product, and contains an **add product** button, when pressed on, the product will be added and we will be directed to the **All posts view**.

The View

Three **views** are required:

1. **All products:** fetches and presents all products. Pressing on any product, will direct us to the **single product view with the id of the product** as a route parameter**.**
2. **A single product:** fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view.**
3. **A create product:** allows creating a new product, and contains an **add product** button, when pressed on, the product will be added and we will be directed to the **All posts view**.



The View - AllProducts.vue

```
<template>
  <div class="AllProducts">
    <div id="Products-list">
    <h1>All Products</h1>
      <ul>
       <div class="item" v-for="product in products" :key="product.id">
         <a class="singleproduct" :href="'/api/aproduct/' + product.id">
           <span class="code"> <b>Code:</b> {{product.code }} </span> <br/>
           <span class="name"> <b>Name:</b> {{product.name }} </span> <br/>
           ...
         </a>
       </div>
      </ul>
    </div>
  </div>
</template>
```

1. **All products:** fetches and presents all products. Pressing on any product, will direct us to the **single product view with the id of the product** as a route parameter**.**

The View - AllProducts.vue

```
<script>
  export default {
  name: "AllProducts",
  data() { return {products:[],};},
  methods: {
    fetchProducts() {
        fetch(`http://localhost:8082/api/products/`)
        .then((response) => response.json())
        .then((data) => (this.products = data))
        .catch((err) => console.log(err.message));
    },  },
  mounted() {
    this.fetchProducts();
    console.log("mounted");
  },};
</script>
```

1. **All products:** fetches and presents all products. Pressing on any product, will direct us to the **single product view with the id of the product** as a route parameter**.**

The View - AProduct.vue

```
<template>
  <div class="A Product">
    <div id="form">
      <h3>A Product </h3>
      <label for="code"> Code : </label>
      <input name="code" type="text" id=" code " required v-model="product.code" />
      <label for="name"> Name : </label>
      <input name="name" type="text" id="name" required v-model="product.name" />
      ...
    </div>
    <div>
      <button @click="updateProduct" class="updateProduct">Update Product</button>
      <button @click="deleteProduct" class="deleteProduct">Delete Product</button>
    </div>
  </div>
</template>
```

2. **A single product:** fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view.**

The View - AProduct.vue

```
<script>
export default {
  name: "AProduct",
  data() {return {product: {code: "", name: "", description: "", price: }, }; },
  methods: {
    fetchAProduct(id) {
      fetch(`http://localhost:8082/api/products/${id}`)
        .then((response) => response.json())
        .then((data) => (this.product = data))
        .catch((err) => console.log(err.message));},
```

2. **A single product:** fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view.**

The View - AProduct.vue

```
 updateProduct() {
      fetch(`http://localhost:8082/api/products/${this.product.id}`, {
        method: "PUT", headers: { "Content-Type": "application/json", },
        body: JSON.stringify(this.product),})
        .then((response) => {console.log(response.data);
          this.$router.push("/api/allproducts"); })
        .catch((e) => {console.log(e); });},
 deleteProduct() {
      fetch(`http://localhost:8082/api/products/${this.product.id}`, {
      ... // to be done in the lab
      },
  mounted() {
      this.fetchAProduct(this.$route.params.id);
  },
};
</script>
```

2. **A single product:** fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view.**

The View - AddProduct.vue

```
<template>
  <div class="form">
    <h3>Add a Post</h3>
    <label for="code">Code: </label>
    <input name="code" type="text" id="code" required v-model="product.code" />
    <label for="name">Name: </label>
    <input name="name" type="text" id="name" required v-model="product.name" />
    ...
    <button @click="addProduct" class="addProduct">Add Product</button>
  </div>
</template>
```

3. **A create product:** allows creating a new product, and contains an **add product** button, when pressed on, the product will be added and we will be directed to the **All posts view**.

The View - APost.vue

```
 updateProduct() {
      fetch(`http://localhost:8082/api/products/${this.product.id}`, {
        method: "PUT", headers: { "Content-Type": "application/json", },
        body: JSON.stringify(this.product),})
        .then((response) => {console.log(response.data);
          this.$router.push("/api/allproducts"); })
        .catch((e) => {console.log(e); });},
 deletePost() {
      fetch(`http://localhost:8082/api/products/${this.product.id}`, {
        method: "DELETE", headers: {"Content-Type": "application/json"},})
        .then((response) => {console.log(response.data);
          this.$router.push("/api/allproducts");})
        .catch((e) => {console.log(e); }); },},
  mounted() { this.fetchAPost(this.$route.params.id);},};
</script>
```

2. **A single product:** fetches and presents a single product based on the passed id. This view also contains **update** and **delete buttons**, when pressed on, the product will be updated/deleted and we will be directed to the **All products view.**

The View - AddProduct.vue

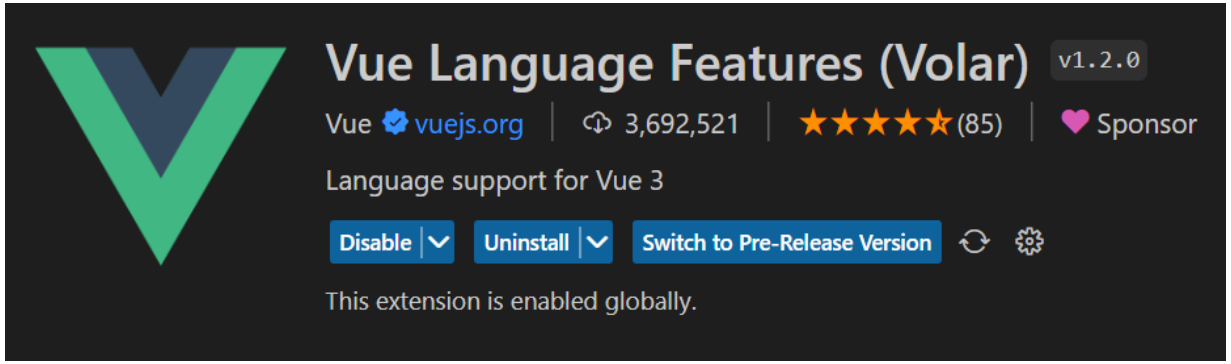
```
<script>
export default {
  name: "AddProduct",
  data() {return { product: {id: "",  code: "",  name: "", ... }, }; },
  methods: {
    addProduct() {
      var data = {id: this.product.is, code: this.product.code, ..., ...};
      fetch("http://localhost:8082/api/products", {
        method: "POST",
        headers: {"Content-Type": "application/json", },
        body: JSON.stringify(data)})
      .then((response) => {console.log(response.data);
        this.$router.push("/api/allproducts"); })
      .catch((e) => {console.log(e);
        console.log("error");});};},};
</script>
```

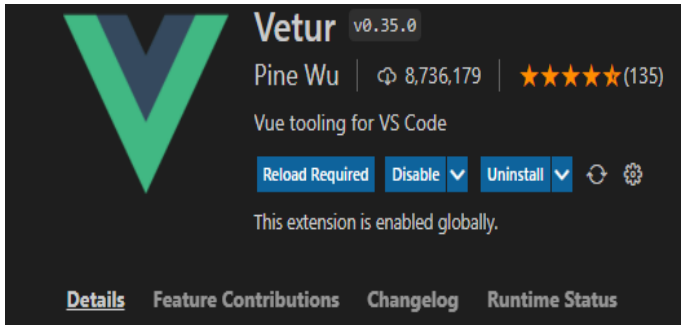
3. **A create product:** allows creating a new product, and contains an **add product** button, when pressed on, the product will be added and we will be directed to the **All posts view**.

Useful Extensions

Useful Extensions

Install the **Volar extension** if you are using **VSCode**, or try to find a similar extension if you are not using **VSCode**. It will be very helpful when coding.

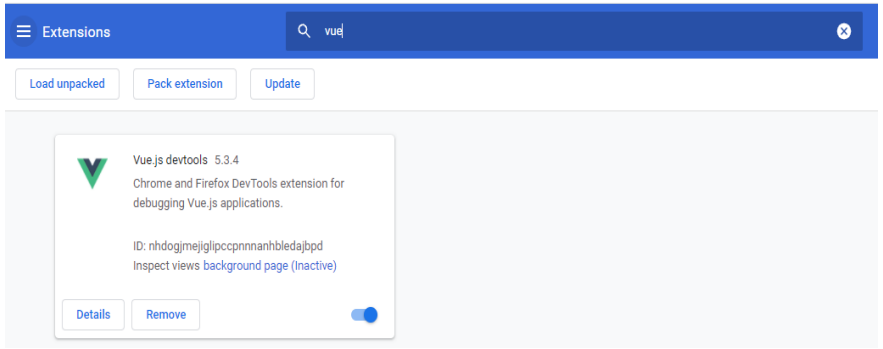


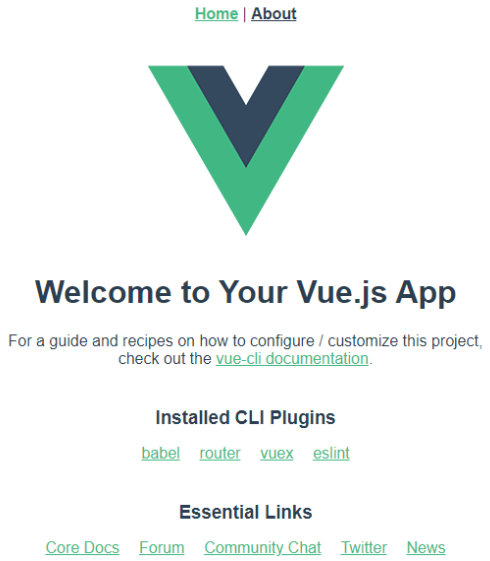


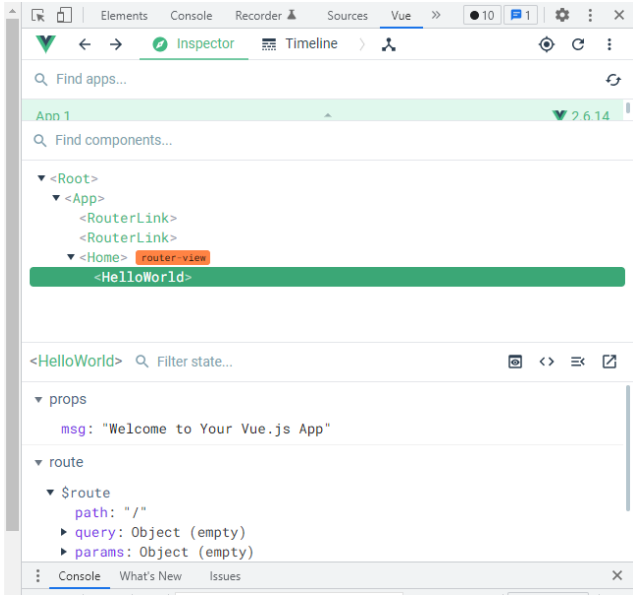
If you were using the **Vetur** extension, you will be suggested to disable it and install Volar.

Useful Extensions

Devtools extension (Google Chrome/ Firefox). If installed successfully, its icon should appear among the icons of extensions in your browser.





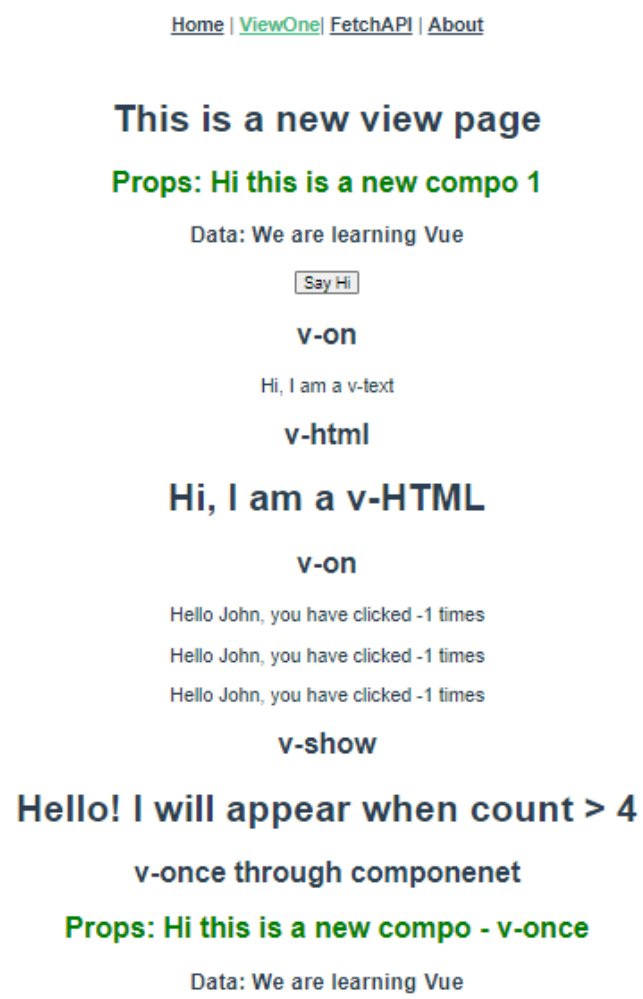


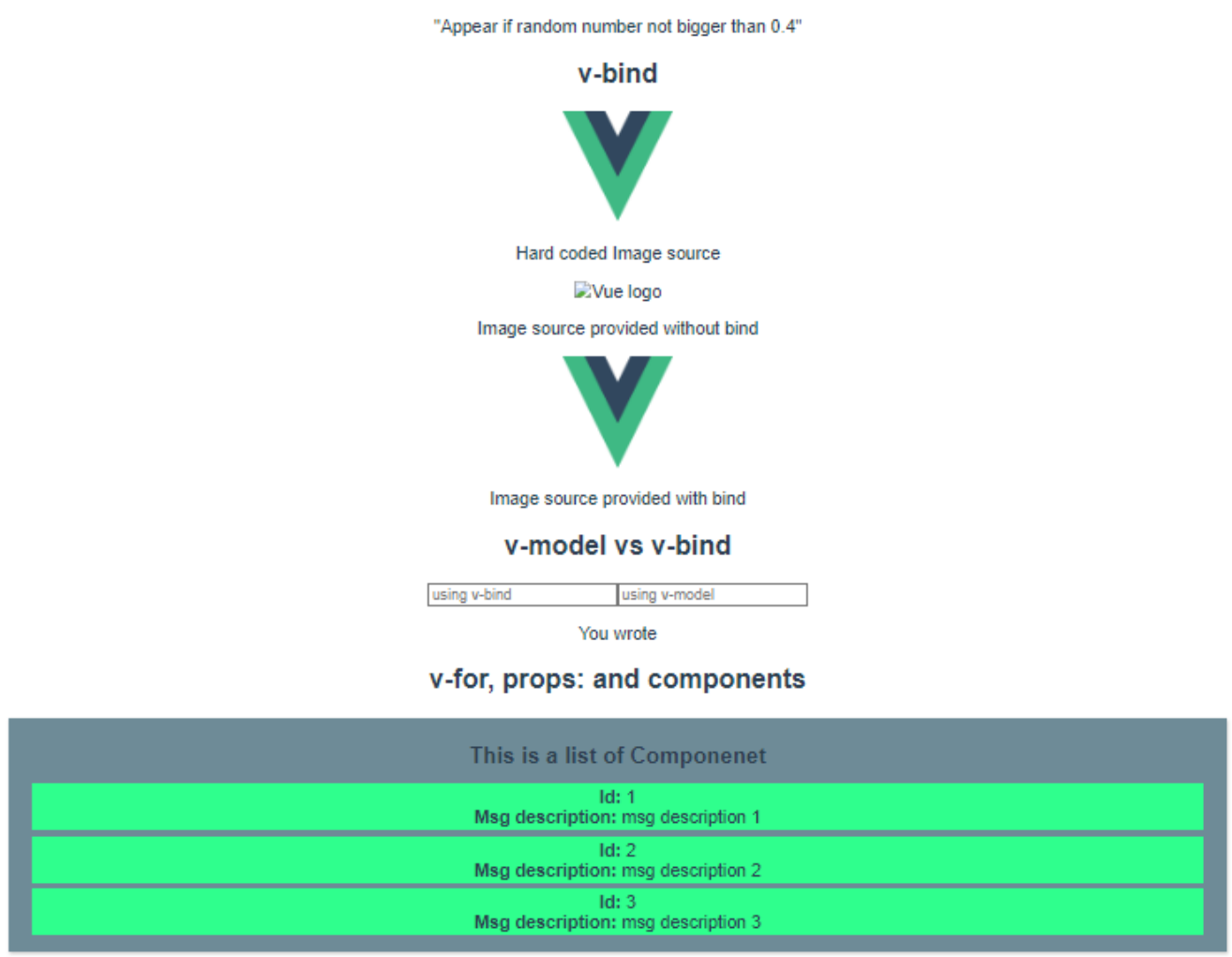
While on the same page, right-click -> inspect and chose the newly added tab, namely the Vue tab.

Vue.js

Try it yourself …

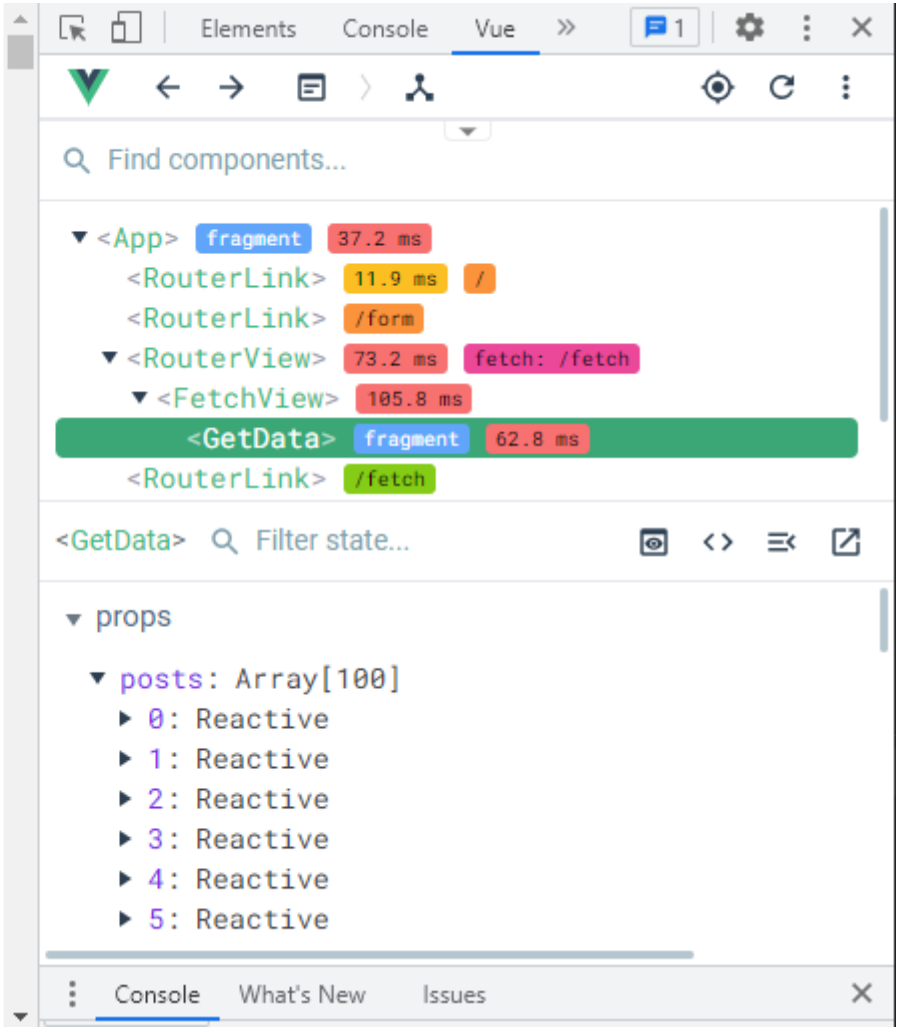
Vue.js – basic





Already available in the shared repo, and the code is annotated to facilitate your understanding, thus, try familiarize yourself with it before the lab.

Vue.js – basic – Simple Fetch API



- A simple example on using Fetch API to fetch and output data in the Vue.js environment.

Extra resources

- **Vue.js – Tutorial** (https://vuejs.org/tutorial/#step-1): an easy step by step tutorial about the essentials of Vue.js
- https://vuejs.org/

UNIVERSITY OF TARTU

Thank You
for your attention

Mohamad Gharib
mohamad.gharib@ut.ee

unitartu

tartuuniversity